

Distribution-Aware Unionable Table Search [Technical Report]

Besat Kassaie
bkassaie@uwaterloo.ca
Data Intelligence Lab
Waterloo, Canada

Renée J. Miller
rjmiller@uwaterloo.ca
Data Intelligence Lab
Waterloo, Canada

Abstract

Satisfying specific data distributions is a fundamental requirement in data-intensive decision making. Data lakes offer a unique opportunity to meet such requirements by bringing together data from diverse sources that may represent different populations, domains, or contexts. While an individual table may not exhibit the desired distribution, appropriately combining multiple tables can potentially achieve a distributional goal. Existing unionable table search, however, focuses primarily on retrieving tables that are highly unionable with a query, without considering the distribution induced by their union. We introduce *Distribution-Aware Unionable Table Search* (DUTS). Given a query table and a distributional requirement over one of its categorical attributes, DUTS retrieves a fixed-size set of unionable tables whose union with the query satisfies the specified distribution while maximizing overall unionability. We investigate two approaches that embody distinct search paradigms: a *unionability-first* approach, which first retrieves highly unionable tables and subsequently attempts to satisfy the distributional requirement, and a *distribution-first* approach, which incorporates the distributional requirement directly into candidate retrieval and selection. The latter uses a novel optimization strategy based on fractional programming. Our experimental evaluation across multiple benchmarks shows that the distribution-first approach more reliably identifies feasible results while preserving high unionability and scales to substantially larger data lakes.

Keywords

Distribution-Aware Data Discovery, Data Lakes, Unionable Table Discovery

1 Introduction

Traditional unionable table search retrieves tables that are semantically compatible with a query table, enabling users to augment a query table with additional tuples. However, semantic compatibility alone is often insufficient. Many real-world applications require retrieved tables to collectively satisfy desired distributional properties over one or more attributes. A data scientist augmenting a training dataset may need to preserve a target class distribution; a healthcare researcher may require patient records whose aggregate demographics match a reference population; a business analyst may demand sales data with a prescribed geographic or product-category breakdown. More broadly, benchmark construction, domain adaptation, and representative data acquisition all impose user-specified distributional constraints that pure semantic similarity cannot capture.

EXAMPLE 1.1 (MOTIVATING EXAMPLE.). Consider a data scientist seeking to augment a training dataset Q (Figure 1) with additional data. Suppose she requires class B to constitute at least 35% of the augmented dataset. Existing table union search system (TUS) would select a set of tables that have the highest unionability with Q . However, it is possible that none of the returned tables individually satisfy the distributional requirement, nor does their union with Q . A straightforward workaround is to increase k , acquire a larger collection of unionable tables, and post-process the result to achieve the desired distribution. Such post-processing could operate at the tuple level, by removing or modifying tuples, or at the table level, by discarding some of the acquired tables. This strategy, however, may be undesirable or even infeasible. First, data providers may license tables as fixed data products and restrict arbitrary modification of their contents. Second, acquiring each table may incur a monetary or operational cost, making over-retrieval followed by post-processing unnecessarily expensive. Ideally, the distributional requirement should therefore be incorporated directly into discovery, allowing the system to retrieve exactly k suitable tables. In our example, selecting T_1 and T_3 yields 40% class- B tuples and satisfies the requirement, although with some loss in unionability. Crucially, the distributional requirement applies to the aggregate result, rather than independently to each candidate. For example, T_1 contains only 10% class- B tuples and therefore does not satisfy the requirement on its own. Nevertheless, T_1 can participate in a feasible solution because its distribution is compensated for by T_3 when the tables are combined with Q . Thus, satisfying the distributional requirement requires reasoning jointly over combinations of tables, rather than independently filtering candidates based on their individual distributions.

This example motivates a more general retrieval problem, which we call *Distribution-Aware Unionable Table Search* (DUTS). Given a query table and a data lake of candidate tables, DUTS retrieves a subset that jointly maximizes unionability while ensuring that the query table and the retrieved tables collectively satisfy a user-specified distributional objective. One obvious instantiation of DUTS is fairness-aware dataset construction, where the distributional objective encodes balance with respect to a protected attribute or set of attributes such as gender or race.

Incorporating distributional requirements into unionable table search introduces three key trade-offs. First, unionability may decrease because satisfying the distributional requirement can exclude otherwise highly unionable tables. Second, enforcing the requirement introduces additional computational overhead. Finally, the search may return an empty result when no feasible solution exists or when existing feasible solutions cannot be found. We wish to control and reduce the impact of these effects on both result quality and runtime, while allowing an empty result when no feasible solution is found.

A substantial body of prior work addresses distributional requirements of datasets by modifying the distribution of an existing dataset post-hoc, either through data transformation [29] adding/deleting tuples to satisfy target criteria while minimizing number of tuples added/removed [25], or through active tuple acquisition guided by acquisition costs [1]. Our work differs fundamentally from these approaches. Rather than correcting distributional skew after data discovery, we incorporate distributional requirements directly into the retrieval process, seeking

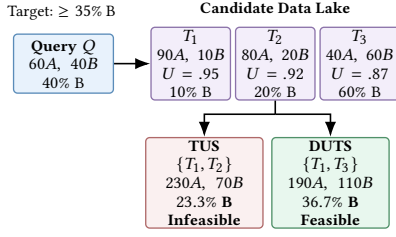


Figure 1: Motivating example with target $\geq 35\%$ class B. Class labels indicate tuple counts, and U denotes unionability with the query.

unionable tables whose data, when aggregated with the query, satisfies the user-specified distribution. This upstream approach can reduce or eliminate the need for downstream data modification, which is particularly advantageous when such modifications are costly, legally constrained, or semantically undesirable. Moreover, unlike tuple-level acquisition frameworks, we operate at the granularity of tables. This setting reflects the practical constraint that individual tuples are typically not available for independent acquisition; instead, data is often acquired at the granularity of entire tables. When table acquisition incurs a cost, as is common in data markets [22, 28], the number of tables that can be acquired under a fixed budget is inherently limited, making distribution-aware table selection particularly important. Finally, our approach is schema-agnostic: we make no assumptions about the existence of metadata, schema, or background knowledge about the data lake’s contents. Together, these distinctions, retrieval-time distribution enforcement, table-level granularity, and schema-agnostic operation, set our work apart from existing approaches to distribution-aware data retrieval. In this paper, we make the following contributions:

- We formalize Distribution-Aware Table Union Search (DUTS, Definition 1), which seeks k tables that maximize unionability with a query table while satisfying a distributional constraint over their *multiset union*. We show that this aggregation-sensitive constraint fundamentally distinguishes DUTS from classical unionable table search and prove that the problem is NP-hard.
- We design and implement three DUTS systems, namely, DUTS-2OPTS, POST-GREEDY, and POST-PREF, each employing a distinct design strategy:
 - DUTS-2OPTS adopts a dual-optimization framework (Figure 2): it first uses fractional programming to construct a distribution-optimal candidate pool. This is the first application of fractional programming and its parametric solution to unionable table search over data lakes. DUTS-2OPTS then solves a linear 0–1 integer program over this reduced pool to select the k tables maximizing total unionability. If the candidate pool contains a distributionally feasible solution, the second-stage formulation is guaranteed to find one.
 - POST-GREEDY and POST-PREF adopt a unionability-first strategy compatible with existing unionable table search systems. The key idea is to repair an initially retrieved, potentially distributionally infeasible result set through table swaps (Section 5). We show that finding a minimum-cost feasible repair is NP-hard and develop two heuristic approaches: POST-GREEDY performs greedy swaps directly, whereas POST-PREF employs preference-based candidate filtering to accelerate the repair process. Unlike DUTS-2OPTS,

these systems provide no feasibility guarantee and may fail to find a feasible result even when one exists.

- We conduct an extensive empirical evaluation of our three DUTS systems, assessing both their effectiveness and efficiency against STARMIE, a high-performing unconstrained unionable table search system.

The remainder of the paper is organized as follows. Section 2 reviews the necessary preliminaries and introduces the notation used throughout the paper. Section 3 presents the problem formulation and provides an overview of our solutions, which are detailed in Sections 4 and 5. Section 6 reports the experimental evaluation.¹ Section 7 summarizes related work, and Section 8 concludes the paper and discusses directions for future work. Detailed proofs and extended results are provided in the Appendix.

2 Preliminaries and Notation

A *table* T consists of a finite multiset of tuples. A *data lake* $\mathcal{T} = \{T_1, \dots, T_{|\mathcal{T}|}\}$ is a repository of such tables. For an attribute A , $\text{Dom}(A)$ denotes its domain, i.e., the set of values that A may take. The values observed in A form a subset of $\text{Dom}(A)$. Given a table T , a distribution attribute $d \in T$ is a categorical attribute of T over which a distributional constraint is to be enforced; d partitions the tuples of T into two groups according to its domain values: target values $\mathcal{V}$ and all other values.

In this work, we focus on unionability scores computed based on attribute alignment between a query table and data-lake tables.² In table union search, TUS, each attribute of a query table Q aligns with at most one attribute of a candidate table T , and not every attribute of Q needs to have a corresponding match in T [4, 13, 24]. In our setting, we additionally require the distribution attribute d of Q to be aligned with a semantically matching attribute of T . We denote by $U_d(Q, T) \in [0, 1]$ the unionability score between Q and T , computed under the constraint that $d \in Q$ is aligned with exactly one semantically matching attribute in T . For a candidate set $S \subseteq \mathcal{T}$, we extend U_d to the set level, as sum per-table score: $U_d(Q, S) = \sum_{T_i \in S} U_d(Q, T_i)$.

Given a table T_i , a distribution attribute $d \in T_i$, and a set of target values $\mathcal{V} \subset \text{Dom}(d)$, let $N(T_i, d, \mathcal{V})$ denote the number of tuples in T_i whose value for d belongs to $\mathcal{V}$. Let $n_i = |T_i|$ denote the number of tuples in T_i , and define the corresponding distributional proportion as $F(T_i, d, \mathcal{V}) = \frac{N(T_i, d, \mathcal{V})}{n_i}$. Let $\tau \in [0, 1]$ denote the minimum required proportion of tuples whose d value belongs to $\mathcal{V}$: $F(T_i, d, \mathcal{V}) \geq \tau$. When clear from context, we omit the parameters for brevity and write N_i and F_i . Given a set of tables S , let $\mathbb{T}(S)$ denote their multiset left-outer-union [16]. For brevity, we write $F_S = F(\mathbb{T}(S), d, \mathcal{V})$. Thus, F_S denotes the proportion of tuples in the union of S whose distribution attribute value belongs to target values. We use F_S throughout rather than introducing a separate ratio function.

We focus on distributional requirements over discrete categorical attributes, such as class labels, gender, or patient age groups, rather than continuous-valued distributions. Accordingly, we require distribution attribute to be categorical. For indexing, we identify candidate categorical attributes using a cardinality threshold: an attribute A of table T is considered categorical if the number of distinct values it takes across the tuples of T is at most θ_{cat} .³

¹Code and datasets are available at <https://github.com/Besatkassaie/DUTS/>

²We acknowledge that not all TUS systems rely on explicit attribute alignment.

³This restriction reflects the application scope of this work rather than a requirement of our distributional constraint; in principle, the constraint can also be defined over higher-cardinality attributes.

Each table T_i with r categorical attributes is represented by r histogram *synopses*, one per attribute. For each categorical attribute $A \in T_i$, its histogram H_i^A maps each value $v \in \text{Dom}(A)$ to its frequency in A , such that $\sum_{v \in \text{Dom}(A)} H_i^A[v] = n_i$.

For any $v \in \text{Dom}(A)$ that does not occur in A , we define $H_i^A[v] = 0$. These synopses are used to efficiently compute F_i for T_i at query time.

3 Problem Definition

Given a user-specified distribution attribute $d \in Q$ and a set of target values $\mathcal{V} \subseteq \text{Dom}(d)$, DUTS aims to identify a set R of k unionable tables from a data lake that maximizes unionability with the query table Q , while ensuring that (i) each retrieved table contains an attribute aligned with d , and (ii) tuples whose aligned attribute takes a value in $\mathcal{V}$ constitute at least a τ fraction of the resulting union. These two requirements distinguish DUTS from traditional unionable table search, which does not enforce attribute alignment or distributional constraints over the retrieved tables. Consequently, if the data lake contains no unionable table with an attribute aligned with $d \in Q$, or if the aligned attributes contain no values from target values, the search returns the empty set.

Because tables in a data lake may use different vocabularies to represent the same concepts, we assume that such differences have been resolved through a unified vocabulary or ontology. Accordingly, the values of the distribution attribute, including the target values $\mathcal{V}$ specified by the user, are consistently represented across datasets. We treat vocabulary alignment as a preprocessing step orthogonal to the distribution-aware retrieval problem studied here.

In this work, we focus on a single attribute for distribution attribute, which captures important practical use cases. For instance, in a classification setting, distribution attribute may represent the class-label attribute (e.g., *diabetic* in Figure 2), while target values specifies the target class values (e.g., *Yes*). This enables a data scientist to retrieve unionable tables that, when combined with the query table, achieve a desired class distribution. Or distribution attribute can represent a group attribute such as race or gender and $\mathcal{V}$ can represent protected groups. We develop our framework accordingly and leave joint constraints over multiple attributes to future work.

DEFINITION 1 (DISTRIBUTION-AWARE UNIONABLE TABLE SEARCH). Given a data lake $\mathcal{T}$, a query table Q , a distribution attribute $d \in Q$, a set of target values $\mathcal{V} \subset \text{Dom}(d)$, a unionability scoring function U_d , and a result size k , the distribution-aware unionable table search problem is defined as finding a set of tables R of size k :

$$R = \arg \max_{S \subseteq \mathcal{T}, |S|=k} U_d(Q, S) \quad (1)$$

where $\tau \in (0, 1]$. s.t. $F_{S \cup \{Q\}} \geq \tau$,

The objective is defined over the unionability, while the feasible solution space is constrained by distributional requirements.

A similar constrained-optimization formulation has been studied in information retrieval. Celis et al. [6] formulate the *constrained ranking maximization* problem, which maximizes ranking utility subject to lower- and upper-bound constraints on the representation of items with specified properties in the retrieved ranking. DUTS follows a similar general principle: maximizing retrieval utility, i.e., unionability, subject to requirements on the composition of the retrieved results. However, rather than constraining the representation of retrieved items at *different ranking*

positions, DUTS maximizes the unionability of a *set* of retrieved tables while requiring tuples with specified attribute values to constitute at least a given fraction of their union.

Note that the distributional constraint in Definition 1 applies to the aggregate result, rather than to individual tables: the aggregated table $\mathbb{T}(S \cup Q)$ must satisfy the specified distributional requirement.

$F_{S \cup \{Q\}}$ (recall this is short-hand for $F(\mathbb{T}(S \cup \{Q\}))$) is inherently *aggregation-sensitive*: The aggregate proportion cannot, in general, be derived from a tables' local properties alone, as it also depends on their relative cardinalities. Consequently, larger tables can disproportionately obscure the distributional characteristics of smaller tables. This distinction is particularly important in data lakes, where unionable tables may vary substantially in size and distribution.

A straightforward solution for finding R requires enumerating all subsets of size k , computing both their distribution and unionability scores, and selecting the subset that optimizes the objective. This exhaustive search is computationally expensive in practice. Next, we show that the problem is, in fact, **NP-hard**.

THEOREM 3.1. *Finding R in Definition 1 is NP-hard.*

The hardness established in Theorem 3.1 does not stem from evaluating the distributional constraint itself, but from maximizing unionability subject to this constraint, which induces the combinatorial structure of an exact- k knapsack problem. This motivates decoupling the two objectives and addressing them independently. We therefore consider two general strategies: (i) first satisfying the distributional requirement and then maximizing unionability through a dual-optimization framework (DUTS-2OPTS) (Section 4); or (ii) a simpler post-processing algorithm (POST) where we first optimize unionability and then enforce the distributional requirement in a post-processing step (Section 5). As our experiments show that the former provides more reliable results, we focus on this strategy and present it in detail next. We describe the latter subsequently for comparison.

4 A Dual-Optimization Framework: DUTS-2OPTS

4.1 Overview and End-to-End Pipeline

DUTS-2OPTS follows a four-step retrieval-and-selection pipeline with two distribution-aware optimization stages (Figure 2).⁴ The pipeline begins with attribute-level retrieval and then transitions to table-level optimization, scoring, and selection.

Index retr. (Attr.) $\rightarrow$ *OPTSTAGE 1 (Tbl.)*
 $\rightarrow$ *Unionability comp. (Tbl.)* $\rightarrow$ *OPTSTAGE 2 (Tbl.)*

Given a query table Q , a distributional attribute d , and a set of target values $\mathcal{V}$, the system first probes an attribute-level index to retrieve candidate attributes that are semantically related to d and contain at least one value from $\mathcal{V}$. Attribute similarity provides a lightweight proxy for table unionability, while overlap with $\mathcal{V}$ filters out candidates that cannot positively contribute toward satisfying the distributional lower-bound constraint. Since we consider the single distribution attribute setting, we make sure that each attribute correspond to exactly one table. We map the attributes to their corresponding tables, forming the table-level candidate pool passed to OPTSTAGE 1. OPTSTAGE 1 uses the distributional information of these tables to select $\alpha \times k$ candidates that

⁴The "2OPTS" in DUTS-2OPTS refers to its two distribution-aware optimization stages.

maximize the achievable distributional proportion, where $\alpha > 1$ is a user-defined expansion factor. At this point, the unionability of each surviving table with Q is computed. OPTSTAGE 2 then solves Problem (1) over this reduced pool of $\alpha \times k$ candidates, rather than the entire repository $\mathcal{T}$ and select the final k tables. This organization delays the more expensive unionability computation and NP-hard optimization until the candidate space has been substantially reduced.

4.2 Attribute-Level Indexing and Candidate Retrieval

The relevance of a data-lake table is determined primarily with respect to the query's distribution attribute d . Accordingly, we use attributes, rather than entire tables, as the retrieval unit. Furthermore, we restrict retrieval to categorical attributes. A table containing no categorical-attribute cannot contribute to the distribution-aware retrieval and is therefore excluded from further consideration.

4.2.1 Two Orthogonal Notions of Relevance. A categorical attribute is considered a plausible match for the query's distribution attribute only if it satisfies both of the following conditions:

- **Semantic relevance.** A candidate attribute should represent the same concept as d , even when the two attributes use different names (e.g., nationality and country_of_origin).
- **Value overlap.** A candidate attribute must share at least one value from the query's target values $\mathcal{V}$. Thus, semantic relevance alone is insufficient: an attribute with no values from $\mathcal{V}$ is discarded because it cannot contribute to the distributional objective.

These two signals capture orthogonal aspects of relevance. Two attributes may be semantically equivalent while sharing no values, or they may share values despite representing different concepts. Consequently, our retrieval procedure evaluates both signals independently and retains only attributes that satisfy both criteria.

4.2.2 Indexing. We leverage existing index structures to efficiently evaluate both relevance signals. Users specify d , $\mathcal{V}$, and τ at query time; these parameters are therefore unavailable during index construction.

Semantic Relevance via HNSW. Each categorical attribute is represented as a fixed-dimensional vector using a pretrained encoder. Our framework is not tied to a particular embedding method; any attribute representation that supports semantic similarity computation can be used. Representations that also capture the context of the source table are particularly beneficial, as they provide an indirect table-level signal during attribute-level retrieval. For example, STARMIE [13] incorporates information from surrounding attributes in the same table when learning an attribute representation, thereby producing contextualized attribute embeddings. We use these embeddings in our implementation.

The attribute embeddings are indexed using a Hierarchical Navigable Small World (HNSW) graph [20]. HNSW is a graph-based approximate nearest-neighbor (ANN) index. HNSW organizes the embedding space into a multi-layer neighborhood graph, enabling efficient retrieval of attributes whose representations are close to that of the query attribute. At query time, we issue a top- n ANN query using the embedding of the query's d and retain only neighbors whose Euclidean distance is below a threshold. We denote the resulting set of attributes by D_{sem} .

Value-overlap Relevance via Posting Lists. Our value-overlap retention rule is a logical predicate rather than a similarity measure: an attribute A is retained if

$$|\mathcal{V} \cap V_A| \geq 1,$$

where V_A denotes the set of observed values of A . This is a Boolean membership test, and we require *every* attribute satisfying it, with no ranking or top- n cutoff involved. We therefore evaluate it exactly, using a classical inverted index. For each observed value v of an indexed attribute $A \in T_j$, the inverted index maintains a posting list: $\text{PL}(v) = \{(T_j, A) : v \in V_A\}$. Given the query's target values $\mathcal{V}$, the index retrieves

$$D_{\text{ovl}} = \bigcup_{v \in \mathcal{V}} \text{PL}(v),$$

which contains *exactly* the (table, attribute) pairs whose attributes satisfy $|\mathcal{V} \cap V_A| \geq 1$.

Note that the inverted index must be over the vocabulary of the data lake and hence very large. However, other work has shown that this is feasible (even for data lakes with vocabulary sizes in the hundreds of millions [30]). Notice, that unlike in joinable table search, we are using the index not to compute the size of two attributes overlap, but merely the existence of an overlap, a much faster operation. In summary, HNSW uses the embedding of Q 's distribution attribute to identify semantically relevant attributes, while the posting-list index uses the user-specified $\mathcal{V}$ to identify attributes with value overlap. The remaining attributes of Q do not participate in retrieval; their contribution is evaluated later in the pipeline, as described in Section 4.4.

Combining Index Retrieval Results. HNSW and the inverted index return sets of (table, attribute) pairs. To ensure that (T_j, A) survives only if the same attribute A satisfies both the semantic and value-overlap criteria, we intersect the two sets: $D_{\text{pair}} = D_{\text{sem}} \cap D_{\text{ovl}}$. Since D_{ovl} is exact, any retrieval uncertainty in D_{pair} arises from HNSW. False positives from HNSW may be removed by the intersection if they fail the exact value-overlap predicate, whereas false negatives cannot be recovered and may therefore reduce the quality of the final solution.

The intersection does not necessarily yield a one-to-one mapping between tables and attributes, since multiple attributes from the same table may satisfy both filters and appear in D_{pair} . We therefore retain, for each table, the qualifying attribute with the highest semantic similarity to the query's distribution attribute. Semantic similarity is used for this selection because value overlap is a Boolean criterion and cannot distinguish among qualifying attributes; ties are broken arbitrarily. This step incurs negligible overhead, as the retrieved sets are expected to be small and requires no additional index probes. We denote the resulting set of (table, attribute) pairs by D .

Metadata Management. For each indexed categorical attribute, we precompute its embedding, observed value set, and histogram synopsis. The embedding and observed value set are maintained by their respective index structures, while the histogram synopsis is stored separately and accessed on demand. For non-categorical attributes, embeddings are stored in a key-value structure keyed by the table-attribute identifier, enabling efficient lookup during table-level unionability computation. This organization is an implementation choice and is independent of any particular underlying system.

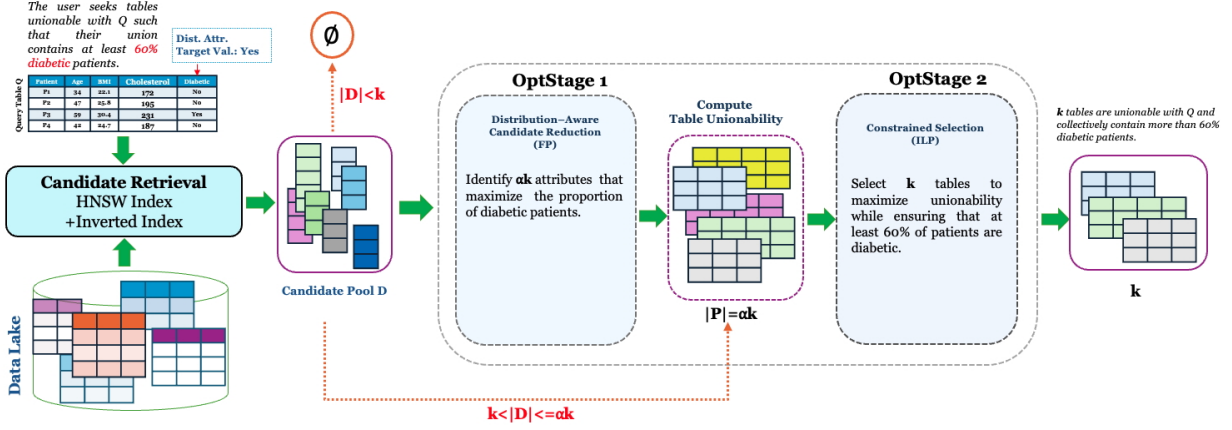


Figure 2: Overview of the DUTS-2OPTS pipeline for identifying k unionable tables satisfying the desired distributional constraint. Candidate processing progresses from attribute-level retrieval to table-level OPTSTAGE 1, unionability computation, and OPTSTAGE 2. If $|D| < k$, the system returns an empty set; if $k \leq |D| \leq \alpha k$, it skips OPTSTAGE 1.

4.3 OPTSTAGE 1: Distribution-Aware Table Selection

Recall that each candidate in D is a pair (T_i, A_i) , where A_i is the attribute of T_i selected as the match for the query's distributional attribute d . OPTSTAGE 1 operates at the table level over $\mathcal{D} = \{T_i \mid (T_i, A_i) \in D\}$. There is a one-to-one mapping between D and $\mathcal{D}$, and thus $|D| = |\mathcal{D}|$. The association between each T_i and its matched attribute A_i is retained to compute the table's distributional contribution during OPTSTAGE 1.

Given the desired candidate pool size $\alpha \times k$, OPTSTAGE 1 seeks a subset $P \subseteq \mathcal{D}$ with $|P| = \alpha \times k$ that maximizes the achieved distributional objective:

$$P = \arg \max_{S \subseteq \mathcal{D}, |S| = \alpha k} F_{S \cup \{Q\}} \quad (2)$$

The expansion factor $\alpha > 1$ controls the trade-off between computational efficiency and solution quality. By maximizing the distributional objective, OPTSTAGE 1 focuses the subsequent optimization (OPTSTAGE 2) on tables that are most promising for satisfying the desired distributional requirement.

Next, we show that maximizing $F_{S \cup \{Q\}}$ can be formulated as a linear-fractional optimization problem over the table-selection variables and solved exactly using Dinkelbach's method [11]. The following observation forms the basis of our optimization. The distributional score of a candidate set $S \subseteq \mathcal{D}$ when unioned with the query table Q can be expressed as:

$$F_{S \cup \{Q\}} = \frac{\sum_{T_i \in S \cup \{Q\}} F_i n_i}{\sum_{T_i \in S \cup \{Q\}} n_i} = \sum_{T_i \in S \cup \{Q\}} w_i F_i, \quad w_i = \frac{n_i}{\sum_{T_i \in S \cup \{Q\}} n_i}.$$

Since $w_i \geq 0$ for all $i \in S \cup \{Q\}$ and $\sum_{i \in S \cup \{Q\}} w_i = 1$, the distributional score $F_{S \cup \{Q\}}$ is exactly a weighted average of the individual table scores F_i , with each table weighted by its cardinality. Consequently, the maximizing subset is *not* simply the $\alpha \times k$ tables with the largest individual F_i : a table with a moderate F_i but favorable n_i can shift the weighted combination more than a table with a higher F_i but small n_i . This is precisely the interaction that ordinary top- $(\alpha \times k)$ ranking by F_i fails to account for, and is what necessitates the approach described next.

4.3.1 Formulation as 0-1 fractional programming. Let $z := |\mathcal{D}|$. We introduce a binary indicator vector $\mathbf{x} = (x_1, \dots, x_z) \in \{0, 1\}^z$, where $x_i = 1$ indicates that candidate table $T_i \in \mathcal{D}$ is selected. The

distributional score of the resulting union can then be expressed as a function of $\mathbf{x}$:

$$F(\mathbf{x}) = \frac{N_Q + \mathbf{N}^\top \mathbf{x}}{n_Q + \mathbf{n}^\top \mathbf{x}} = \frac{N_Q + \sum_{i=1}^z N_i x_i}{n_Q + \sum_{j=1}^z n_j x_j},$$

where $\mathbf{N} = (N_1, \dots, N_z)$ and $\mathbf{n} = (n_1, \dots, n_z)$, and $(\cdot)^\top$ denotes the transpose operator. Both the numerator and denominator are linear functions of $\mathbf{x}$. Consequently, $F(\mathbf{x})$ is a linear-fractional objective over binary decision variables, yielding a 0-1 fractional program. Our optimization problem can be expressed as:

$$\max_{\mathbf{x} \in \{0,1\}^z} F(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{1}^\top \mathbf{x} = \alpha \times k,$$

i.e., maximize this linear-fractional objective subject to the cardinality constraint that exactly $\alpha \times k$ entries of $\mathbf{x}$ equal 1. $F(\mathbf{x})$ does not decompose into independent per-table contributions: the effect of including table i depends on the other tables already selected in S , as its inclusion changes both the numerator and denominator jointly.

Linear fractional programs were first solved using a linear programming transformation proposed by Charnes and Cooper [8]. Later, Dinkelbach [11] introduced a parametric algorithm that has been shown to be highly effective in practice [21, 27]. We adopt Dinkelbach's method in this work.

Dinkelbach's parametric method. Dinkelbach's method introduces an auxiliary parameter $\lambda \in \mathbb{R}$ to transform a fractional objective into a sequence of parametric problems. Adapting this transformation to our setting, for each candidate table $T_i \in \mathcal{D}$, we define $y_i(\lambda) = N_i - \lambda n_i$. For a fixed λ , we then consider the following parametric objective:

$$\Phi(\lambda) = N_Q - \lambda n_Q + \max_{|S| = \alpha k} \sum_{i \in S} y_i(\lambda).$$

This subtraction of the denominator, scaled by λ , from the numerator converts the original *ratio* objective $F(\mathbf{x})$ into a *linear* objective in $\mathbf{x}$, for a fixed value of λ . Because the objective $\sum_{i \in S} y_i(\lambda)$ is now additively separable across tables, its maximizer over subsets of size exactly $\alpha \times k$ is trivial to compute: it is simply the $\alpha \times k$ tables with the largest values of $y_i(\lambda)$, obtainable via a single top- $(\alpha \times k)$ selection.

The classical result underlying Dinkelbach's method [11] is that the optimal achievable proportion is exactly the unique root

of Φ :

$$\Phi(\lambda) = 0 \iff \lambda = \max_{|S|=\alpha \times k} F_{S \cup \{Q\}}.$$

Thus, instead of directly optimizing the fractional objective, Dinkelbach's method iteratively solves a simpler problem. In our setting, each such problem reduces to selecting the $\alpha \times k$ tables with the largest scores $y_i(\lambda)$. Next, we describe our adaptation of Dinkelbach's iterative method.

The iterative procedure. Root-finding on Φ is carried out by the following iteration:

- (1) Set $t = 0$ and initialize $\lambda^{(t)}$ (e.g., the proportion achieved by the full candidate pool, $\lambda^{(t)} = \mathbf{N}^\top \mathbf{1} / \mathbf{n}^\top \mathbf{1}$).
- (2) At iteration t , compute $y_i(\lambda^{(t)}) = N_i - \lambda^{(t)} n_i$ for all i , and let $S^{(t)}$ be the $\alpha \times k$ tables with the largest $y_i(\lambda^{(t)})$.
- (3) Update $\lambda^{(t+1)} = F_{S^{(t)}} = (N_Q + \sum_{i \in S^{(t)}} N_i) / (n_Q + \sum_{i \in S^{(t)}} n_i)$, i.e., the proportion actually achieved by the subset just selected.
- (4) If $\lambda^{(t+1)} = \lambda^{(t)}$ (or the change falls below a numerical tolerance), terminate: $S^{(t)}$ is the exact global optimum and $\lambda^{(t)} = \max_{|S|=\alpha \times k} F_{S \cup \{Q\}}$; this optimum is exactly P of Equation (2). Otherwise, increment t and repeat from Step 2.

Because $\Phi(\lambda)$ is strictly decreasing and convex, Dinkelbach's iteration converges monotonically to its unique root, corresponding to the optimal value of the fractional objective.

At each iteration, the candidate tables are sorted by $y_i(\lambda)$ to select the top $\alpha \times k$ tables, requiring $O(n \log n)$ time for $n = |\mathcal{D}|$ candidates. Thus, over I iterations, the overall time complexity is $O(I \times n \log n)$. In practice, Dinkelbach's method has been shown to converge in only a few iterations [21, 27].

4.4 Unionability Computation via Constrained Attribute Alignment

To compute table-level unionability, we build on TUS methods that establish correspondences between attributes of the query and candidate tables, a process known as *attribute alignment*, as employed by STARMIE [13]. For each candidate table $T_i \in P$, however, the correspondence between its matched attribute A_i and the query's distributional attribute d has already been established. We therefore fix (d, A_i) in the alignment and optimize only the correspondences among the remaining attributes of Q and T_i .

Constrained Alignment. We adapt STARMIE's attribute-alignment procedure [13] to incorporate the correspondence (d, A_i) established during retrieval. Rather than allowing this pair to be reconsidered during matching, we fix it in the alignment and apply bipartite matching only to the remaining attributes.

Specifically, let X and Y denote the attributes of the query table Q and candidate table $T_i \in P$, excluding d and A_i , respectively. We construct a weighted bipartite graph $G = (X \cup Y, E)$, where each edge $(x, y) \in E$ is weighted by the pairwise semantic similarity $s(x, y)$. Following STARMIE, we retain only edges with $s(x, y) \geq 0.6$ and compute a maximum-weight bipartite matching M^* over G . The final alignment consists of the fixed pair (d, A_i) together with the matched pairs in M^* .

Unionability Computation The unionability score between Q and T is defined as the sum of the similarities of all aligned

attribute pairs, including the fixed distributional-attribute alignment:

$$U_d(Q, T) = s(d, A_i) + \sum_{(x, y) \in M^*} s(x, y).$$

Later in the paper, we use a variant of the STARMIE search system in which its original attribute alignment and unionability score computation are replaced by our constrained approach, while preserving its efficient, sublinear retrieval procedure. We refer to this variant as STARMIEC throughout the remainder of the paper.

4.5 OPTSTAGE 2: Unionability-Aware Table Selection

At this point, the unionability score of every table in P has been computed. OPTSTAGE 2 solves Problem (1) over P rather than the full repository $\mathcal{T}$. Restricting the search to P therefore focuses the expensive unionability optimization on candidates that are most promising with respect to the distributional objective. Intuitively, tables that better satisfy the distributional objective are more likely to participate in feasible, high-quality solutions. This restriction does not alter the problem's NP-hardness; it only reduces the search space. Specifically, given P , it selects a subset $R \subseteq P$ with $|R| = k$ that maximizes total unionability while satisfying the distributional constraint:

$$\begin{aligned} R = \arg \max_{S \subseteq P, |S|=k} U_d(Q, S) \\ \text{s.t. } F_{S \cup \{Q\}} \geq \tau. \end{aligned} \quad (3)$$

The objective is a linear sum of unionability scores, while the distributional constraint is fixed.

4.5.1 Solution via Integer Linear Programming (ILP). Defining $y_i(\tau) = N_i - \tau n_i$, which can be precomputed once per candidate, the distributional constraint becomes

$$N_Q - \tau n_Q + \sum_{T_i \in S} y_i(\tau) \geq 0.$$

Introducing binary decision variables $x_i \in \{0, 1\}^{|P|}$, we rewrite Equation (3) as:

$$\begin{aligned} \max_{x_i \in \{0, 1\}^{|P|}} \quad & \sum_{T_i \in P} U_d(Q, T_i) x_i \\ \text{s.t. } \quad & N_Q - \tau n_Q + \sum_{T_i \in P} y_i(\tau) x_i \geq 0, \\ & \sum_{T_i \in P} x_i = k. \end{aligned} \quad (4)$$

The formulation consists of a linear objective and two linear constraints over binary decision variables. Since OPTSTAGE 2 operates over the $\alpha \times k$ tables selected by OPTSTAGE 1, Equation (4) contains only $\alpha \times k$ binary decision variables, independent of the size of the data lake $\mathcal{T}$. This reduced problem size makes the use of a standard ILP solver practical. To avoid unnecessary ILP solves, we first solve the computationally cheaper LP relaxation, obtained by replacing the binary constraints $x_i \in \{0, 1\}$ with $x_i \in [0, 1]$. If the relaxation is infeasible, the ILP is necessarily infeasible, and thus no size- k subset of P satisfies the distributional constraint. Otherwise, we solve the ILP to obtain the exact optimal subset R .

5 POST: Swap-Based Distributional Repair

An alternative design is to first optimize for unionability and then repair, *post hoc*, the resulting set to satisfy the distributional constraint. This approach is appealing because it can operate directly on the candidate set returned by an existing efficient TUS

system. Restricting the subsequent optimization to this candidate set reduces the search space from the entire data lake to a manageable number of tables. However, this design makes solution quality dependent on the initial retrieval: if tables required for a distributionally feasible solution are absent from the retrieved candidate set, the subsequent optimization cannot recover them. We use STARMIEC as the backend search system for the POST-based methods introduced in this section. Hence, every retrieved unionable table has an attribute aligned with the query's distribution attribute; we assume this alignment throughout the remainder of the section without restating it explicitly.⁵

Given a set of $\alpha \times k$ unionable tables returned by STARMIEC, we partition the search space into two disjoint components: (i) a result set R of size k , containing the tables with the highest unionability scores, and (ii) a candidate pool C containing the remaining tables, whose unionability scores are no greater than those of the tables in R . When the union of R with Q violates the distributional constraint, our objective is to replace tables in R with candidates from C such that the resulting union satisfies the constraint while minimizing the loss in unionability.

We next show that when the query table itself satisfies the distributional requirement, R must contain at least one table that individually violates it. This observation motivates restricting the repair heuristic to the violating subset $R_{\text{sw}} \subseteq R$, reducing the search space. This restriction is heuristic: a feasible repair may in principle involve replacing a table outside R_{sw} .

CLAIM 1 (LOCALITY OF VIOLATIONS). *Let Q be a query table with distribution attribute d , target values $\mathcal{V}$, and distributional lower bound τ . Let R be a set of unionable tables. If Q individually satisfies the distributional requirement, $F_Q \geq \tau$, but $F_{R \cup \{Q\}} < \tau$, then there exists at least one table $T_j \in R$ such that $F_j < \tau$.*

Claim 1 enables a conditional reduction of the swap search space. When $F_Q \geq \tau$, we restrict replacement to the individually violating tables $R_{\text{sw}} \subseteq R$. When $F_Q < \tau$, the query itself may cause the aggregate violation, and Claim 1 no longer applies. We therefore conservatively consider all tables in R as potential replacement targets.

Repairing a Violating Union. Given a result set R whose union with Q violates the distributional constraint, we seek to restore feasibility by replacing some tables in R_{sw} with candidate tables. Since candidates, by construction, are no more unionable than the selected tables, every repair might trade unionability for improved distributional balance.

A repair selects a subset $A \subseteq R_{\text{sw}}$ of swappable tables to remove and an equally sized subset $B \subseteq C$ of candidate tables to insert. The repaired result is

$$R' = (R \setminus A) \cup B,$$

where $|A| = |B|$. Consequently, $|R'| = |R| = k$. The repair cost is the loss in unionability,

$$L(A, B) = U_d(Q, R) - U_d(Q, R').$$

DEFINITION 2 (CHEAPEST REPAIR). *Given a distributionally infeasible solution R , a swappable subset $R_{\text{sw}} \subseteq R$, a candidate pool C , and a target threshold τ , find subsets $A \subseteq R_{\text{sw}}$ and $B \subseteq C$ such that $|A| = |B|$ minimizing*

$$L(A, B)$$

⁵This choice is made for implementation convenience and is not inherent to our approach; any TUS system can serve as the backend, provided that it supports the required alignment of distribution attribute.

	U	N	$n - N$	n	F
t_1	1	1	9	10	.10
t_2	1	3	7	10	.30
c_1	.8	2	0	2	1.0
c_2	.9	4	2	6	.67
c_3	.95	14	6	20	.70

	N	n	N/n
t_1	0	10	.00
t_2	15	25	.60
c_1	4	4	1.0
c_2	5	5	1.0
c_3	16	20	.80

(a) Greedy subset selection fails

(b) Greedy incompleteness

Figure 3: Tables used in Examples 5.1 and 5.2. U denotes the unionability of the tables.

subject to

$$F_{R' \cup \{Q\}} \geq \tau,$$

where

$$R' = (R \setminus A) \cup B.$$

Next, we show by example that no local ranking of candidates is sufficient to identify an optimal repair.

EXAMPLE 5.1. *Let $\tau = 0.75$, $R_{\text{sw}} = \{t_1, t_2\}$, $C = \{c_1, c_2, c_3\}$, as in Figure 3a. Selecting candidates greedily by unionability chooses $B = \{c_2, c_3\}$, whose pooled ratio is*

$$\frac{14 + 4}{20 + 6} = \frac{18}{26} \approx 0.69 < 0.75.$$

Selecting greedily by individual distribution instead chooses $B = \{c_1, c_3\}$, giving

$$\frac{2 + 14}{2 + 20} = \frac{16}{22} \approx 0.73 < 0.75.$$

The only feasible repair is obtained by selecting $B = \{c_1, c_2\}$, whose pooled distribution satisfies

$$\frac{2 + 4}{2 + 6} = \frac{6}{8} = 0.75.$$

Example 5.1 illustrates feasibility depends on the collective effect of the selected candidates, requiring the optimization to search over candidate subsets rather than make independent greedy choices.

THEOREM 5.1. *Computing the cheapest repair problem is NP-Hard.*

5.1 A Greedy Repair Heuristic

Theorem 5.1 rules out an efficient algorithm for computing the minimum-cost repair in the worst case. We therefore adopt a greedy heuristic that trades optimality for tractability. The heuristic processes violating tables sequentially and, at each step, commits to a swap that yields the greatest improvement in distributional satisfaction, at the cost of a potential reduction in unionability. Importantly, it never performs a swap that worsens the distributional constraint. A natural next question is whether the candidates in C can be ranked once, allowing the most promising replacements to be considered first and potentially accelerating the search. Claim 2 shows that a global ranking is insufficient: the best candidate depends on the specific table being replaced. Consequently, candidates must be evaluated separately for each swappable table.

CLAIM 2 (TABLE-DEPENDENT OPTIMAL REPLACEMENT). *For a swappable table $t \in R_{\text{sw}}$ and candidate $c \in C$, let $R(t, c) = ((R \cup \{Q\}) \setminus \{t\}) \cup \{c\}$*

$$c^*(t) = \arg \max_{c \in C} F_{R(t, c)}$$

There may exist $t, t' \in R_{\text{sw}}$ such that $c^(t) \neq c^*(t')$.*

Since each swap may reduce unionability, we seek, at every step, a replacement that yields the largest improvement in distributional balance at the smallest possible unionability cost, minimizing both the number of swaps performed and the total loss incurred. Considering these findings, we propose Algorithm 1, with the default setting for *flt*, to achieve the desired distributional ratio.

Algorithm 1: GREEDY-SWAP

Input: Q : query; R : top- k result; C : candidates; τ : threshold; $\mathcal{V}; d; flt$: preference filtering (default: false)

Output: R : distributionally balanced unionable tables

Precond : $F_{RU\{Q\}} < \tau$

```

1 if  $F_Q > \tau$  then
2    $R_{sw} \leftarrow \text{GETVIOLATING}(R)$ ;
3 else
4    $R_{sw} \leftarrow R$ ;
5  $R_{sw}.\text{SORT}()$ ;
6 if flt then
7    $C \leftarrow \text{CANDIDATEFILTER}(C)$ ;
8 foreach  $t \in R_{sw}$  do
9    $b \leftarrow \text{FINDBEST}(C, t)$ 
10  if  $F_{RU\{Q\}} < F_{R(t,b) \cup \{Q\}}$  then
11     $R \leftarrow R(t, b)$ ;
12     $C \leftarrow C \setminus \{b\}$ ;
13  if  $F_{RU\{Q\}} \geq \tau$  then
14    return  $R$ 
15 return  $\emptyset$  // No feasible result found

```

Three components of Algorithm 1 require further clarification:

- **Sorting violating tables (Line 5).** The tables in R_{sw} are sorted in descending order by *remove-only* distributional gain, $F_{RU\{Q\} \setminus \{t\}} - F_{RU\{Q\}}$, i.e., the improvement in F achieved by removing the table from the result set. Ties are broken by the negative of the table's unionability score, prioritizing tables whose removal costs the least.
- **Selecting the replacement (Line 9).** For each swappable table $t \in R_{sw}$, **FINDBEST** evaluates every candidate in C and returns the one yielding the largest value of F after the swap, i.e., $c^*(t)$ from Claim 2. Ties are broken by minimizing unionability loss.
- **Committing the swap (Line 11).** A swap is committed only if it strictly increases $F_{RU\{Q\}}$; A candidate that does not strictly improve the current distributional objective is discarded, as applying the swap would incur a potential loss in unionability without providing any distributional benefit.

PROPOSITION 5.2. *Algorithm 1 is sound and runs in $O(|R_{sw}| \cdot |C|)$ time.*

We note that Algorithm 1 is sound but not optimal or complete: because it commits to each swap greedily and never reconsiders an earlier decision, it may fail to find a feasible repair even when one exists.

EXAMPLE 5.2. Let $\tau = 0.90$, $R_{sw} = \{t_1, t_2\}$, and $C = \{c_1, c_2, c_3\}$, as shown in Figure 3b, with Q satisfying the distributional constraint and omitted for brevity. Initially $F_{R_{sw}} = 15/35 \approx 0.43$. A feasible repair exists: replacing both t_1 and t_2 with $\{c_1, c_2\}$ gives $F = 9/9 = 1 \geq \tau$. Algorithm 1, however, fails to find it. Processing t_1 first, **findBest** evaluates all three candidates and selects c_3 , since it yields the largest immediate improvement, $F_{R(t_1, c_3)} = 31/45 \approx 0.69$, compared to $19/29 \approx 0.66$ for c_1 and $20/30 \approx 0.67$ for c_2 . This swap is committed, leaving $C = \{c_1, c_2\}$ and $F \approx 0.69 < \tau$. Processing t_2 next, the best remaining choice is c_2 , giving $F_{\{c_3, c_2\}} = 21/25 = 0.84$, still short of τ . The algorithm terminates and returns $\emptyset$, even though the feasible repair $\{c_1, c_2\}$ was available throughout. The failure arises because c_3 offers the largest immediate gain when paired with t_1 alone, but its size ($n = 20$) dilutes any subsequent pairing; the two small, perfectly pure candidates c_1, c_2 only reveal their value in combination, a form of interaction the greedy, one-step-lookahead selection cannot detect.

5.2 Preference-Based Filtering

When the query table itself satisfies the distributional constraint, restricting the search to R_{sw} reduces the cost to $O(|R_{sw}| \cdot |C|)$. Nevertheless, this remains costly when either set is large, as is common in data lakes with thousands of candidate tables. We therefore introduce a filtering step that prunes C prior to the main loop, reducing the number of candidates each call to **FINDBEST** must evaluate without sacrificing solution quality. As discussed in Claim 2, the ordering of tables in the candidate pool generally depends on the particular swappable table being replaced. However, some conditions hold independently of the specific swappable table and can therefore be used to prune candidates from C in advance. To formalize these conditions, we use the notion of *preferences* [10].

5.2.1 Preference Formulas. Chomicki [10] provides a framework for expressing preferences over tuples. A preference is specified by a relation $C(t_1, t_2)$, where $t_1 \succ t_2$ means that t_1 is preferred to t_2 according to C . A tuple t is *dominated* if another tuple t' is preferred to it ($t' \succ t$); otherwise, it is *non-dominated*. The **WINNOW** operator $\omega_C(r)$ filters a relation r by removing dominated tuples: $\omega_C(r) = \{t \in r \mid \nexists t' \in r : t' \succ t\}$.

EXAMPLE 5.3. Consider four candidate tables for replacing a table $t \in R_{sw}$, with the following distributional statistics stored in r :

	$N(c_i)$	$n(c_i) - N(c_i)$
c_1	60	20
c_2	50	30
c_3	55	25
c_4	45	15

Consider the preference relation $C(c_i, c_j)$:

$$N(c_i) > N(c_j) \wedge n(c_i) - N(c_i) < n(c_j) - N(c_j)$$

Here, $c_1 \succ c_2$ and $c_1 \succ c_3$: compared with each, c_1 contributes more matching tuples and fewer non-matching tuples. Hence, for any fixed replacement target t , replacing t with c_1 achieves a higher distributional score than replacing it with either c_2 or c_3 . The two candidates can therefore be safely discarded. In contrast, c_1 and c_4 are incomparable: c_1 contributes more matching tuples, whereas c_4 contributes fewer non-matching tuples. $\omega_C(r)$ therefore retains $\{c_1, c_4\}$ and eliminates $\{c_2, c_3\}$. So this filtering does not alter the best replacement.

The key property that makes **WINNOW** useful for pruning is its safety guarantee: any dominated tuple can be discarded without affecting the optimal answer. Chomicki proposes several algorithms for computing $\omega_C(r)$, including a nested-loop algorithm, (Algorithm 2), that is correct for any preference relation without requiring additional conditions. We exploit this framework

Algorithm 2: CANDIDATEFILTER(C)**Input:** Candidate set C **Output:** Filtered candidate set C'

```

1  $C' \leftarrow C$ ;
2 foreach  $c_i \in C$  do
3   foreach  $c_j \in C$  do
4     if  $c_j \succ c_i$  then
5        $C' \leftarrow C' \setminus \{c_i\}$ ;
6     break;
7 return  $C'$ ;

```

directly: by instantiating $\succ$ as the dominance relation over candidate tables (Equation (5)), we prune C to a smaller pool C' of non-dominated candidates, with the guarantee that no pruned candidate can be part of a repair *found* by Algorithm 1. Let for brevity $F_t(c) := F_{R(t,c) \cup \{Q\}}$. Given two candidate tables c_1 and c_2 , and an arbitrary swappable table $t \in R_{sw}$ to be replaced, we define the preference relation over candidate tables as follows:

$$\begin{aligned}
 c_1 \succ c_2 \iff & \left[F_{R(t,c_1) \cup \{Q\}} > F_{R(t,c_2) \cup \{Q\}} \right. \\
 & \wedge U_d(Q, c_1) \geq U_d(Q, c_2) \Big] \\
 & \vee \left[F_{R(t,c_1) \cup \{Q\}} = F_{R(t,c_2) \cup \{Q\}} \right. \\
 & \wedge U_d(Q, c_1) > U_d(Q, c_2) \Big].
 \end{aligned} \quad (5)$$

We next establish conditions under which $F_{R(t,c_1) \cup \{Q\}} > F_{R(t,c_2) \cup \{Q\}}$ holds independently of the table t being replaced; that is, conditions under which c_1 is always preferred to c_2 in Algorithm 1 (Line 9), regardless of the choice of t .

CLAIM 3. *Given two candidate tables c_1 and c_2 , and an arbitrary swappable table $t \in R_{sw}$, the following conditions hold:*

$$\begin{aligned}
 N(c_1) > N(c_2), n(c_1) - N(c_1) < n(c_2) - N(c_2) \\
 \implies F_{R(t,c_1) \cup \{Q\}} > F_{R(t,c_2) \cup \{Q\}}, \\
 N(c_1) > N(c_2), n(c_1) - N(c_1) = n(c_2) - N(c_2) \\
 \implies F_{R(t,c_1) \cup \{Q\}} > F_{R(t,c_2) \cup \{Q\}}, \\
 N(c_1) = N(c_2), n(c_2) - N(c_2) > n(c_1) - N(c_1) \\
 \implies F_{R(t,c_1) \cup \{Q\}} > F_{R(t,c_2) \cup \{Q\}}, \\
 N(c_1) = N(c_2), n(c_1) - N(c_1) = n(c_2) - N(c_2) \\
 \implies F_{R(t,c_1) \cup \{Q\}} = F_{R(t,c_2) \cup \{Q\}}.
 \end{aligned}$$

Applying Claim 3, we can prune dominated candidates from C before invoking the main swap loop. Without imposing any additional structure on the preference formula or the preference relation $\succ$, we adopt the classical nested-loop dominance filter [10] (Algorithm 2), which removes any candidate c_i for which there exists a c_j with $c_j \succ c_i$. We enable preference-based filtering by calling Algorithm 1 with *flt* set to true.

Although the filter requires $O(|C|^2)$ comparisons in the worst case, this cost is incurred only once as a preprocessing step, while the pruned pool C' is reused across all $|R_{sw}|$ iterations of Algorithm 1. When $|C|$ is large, the reduction in candidate pool size can therefore yield a substantial net saving.

In our experiments, we implemented two POST-based systems, both using STARMIEC as the backend search system and Algorithm 1 to satisfy the desired distributional ratio. We denote the variant with *flt* = False as POST-GREEDY and that with *flt* = True as POST-PREF.

6 Experiments

We first evaluate the *effectiveness* of DUTS-2OPTS, POST-GREEDY, and POST-PREF and compare them against the unconstrained

STARMIE baseline. Based on these results, we then focus our *scalability* evaluation on DUTS-2OPTS, which provides the strongest effectiveness. Finally, we conduct an *ablation study* to quantify the contribution of its key design choices.

6.1 STARMIE

STARMIE [13] is a learning-based TUS system that serves three roles in our work. STARMIE employs contrastive learning to learn contextualized attribute representations from unlabeled tables in a self-supervised manner. By incorporating the context of an attribute within its table, these representations capture semantic of attributes effectively and have been shown to support effective unionable table retrieval. We adopt STARMIE primarily for its demonstrated effectiveness, implementation convenience, and adaptability to constrained alignment; however, our proposed framework is not inherently tied to STARMIE.

First, we use STARMIE's representation model to generate contextualized attribute embeddings for semantic similarity computation. This choice is not fundamental to our approach: any attribute-level representation that supports semantic similarity can be used instead. Second, we use the original STARMIE retrieval system as our unconstrained baseline, since it retrieves tables based on unionability without considering distributional constraints. Third, as discussed earlier, we modify STARMIE's attribute-alignment component to construct STARMIEC, which we use as a backend TUS system for POST-GREEDY and POST-PREF. Details of the STARMIE parameters are provided in Appendix A.2.

6.2 Datasets

We use different benchmarks for different evaluation purposes. However, across all datasets, we use the same threshold for identifying categorical attributes, $\theta_{cat} \leq 50$.

6.2.1 Effectiveness Benchmark. To ensure the existence of distributionally satisfying candidates, we construct a synthesized variant of each benchmark. For each query table, we select a distribution attribute and target values and identify all unionable data-lake tables containing an attribute aligned with distribution attribute. We then augment each such table by duplicating tuples containing target values until their relative frequency reaches at least 0.4. We choose this threshold because our effectiveness experiments target $\tau \geq 0.3$. Thus, every unionable table has a corresponding augmented version capable of satisfying the target distributional requirement.

We select distribution attribute and target values during pre-processing using a fixed random seed. The augmentation is stochastic: after duplicating the minimum number of qualifying tuples required to reach the threshold, we allow a bounded random overshoot, producing varying but valid distributions across tables. Because augmentation only duplicates existing tuples without modifying attributes or values, it preserves the underlying unionability relationships. We apply the same procedure to all query tables containing categorical attributes to avoid systematically biasing the retrieved distributions through the query tables themselves. We use three datasets with available unionability ground truth and construct synthesized variants of each: SANTO-SMALL [17], and TUS-SMALL and TUS-LARGE [24].

6.2.2 Scalability Benchmark. We evaluate the scalability of DUTS-2OPTS using two datasets. We use the SANTOS-LARGE benchmark [17], which contains 80 query tables. Of these, 78 queries contain at least one categorical attribute and are eligible for our

Table 1: Summary of the benchmarks used in our evaluation.

Experiment	Benchmark	# Queries	# Tables
Effectiveness	SANTO-SMALL	48	999
	TUS-SMALL	92	9,178
	TUS-LARGE	142	18,309
Scalability	SANTOS-LARGE	46	11,086
	WDC-10K	67	10K
	WDC-100K	67	100K
	WDC-1M	67	1M

experiments. The distribution attributes and target values selection strategy used in the effectiveness experiments yields too few candidates for a meaningful scalability evaluation. We therefore adopt a different strategy for selecting distribution attributes, target values, and τ , with the goal of retaining sufficiently large candidate pools to exercise all stages of the pipeline, thereby avoiding early termination or bypassing OPTSTAGE 1 (Figure 2). Details of this selection procedure are provided in Appendix A.3.

We pick 46 queries for which feasible solutions exist across all configurations with $k \in \{10, 15\}$, $\alpha \in \{2, 3, 4\}$, and $\tau = 0.052$.

Furthermore, we also use the Web Tables corpus (WDC) [19]. We construct three nested data-lake tiers containing 10K, 100K, and 1M tables, with each tier contained in the next. We evaluate scalability on these tiers and use the observed growth trends to infer the expected cost for the full corpus. Web tables are generally small (median of six rows), causing many attributes to satisfy our categorical criterion. As with SANTOS-LARGE, we select queries for which a distributionally feasible solution exists. Since Web Tables does not provide a predefined query workload, we randomly sample an initial pool of 210 tables and apply the same feasibility-based selection procedure used for SANTOS-LARGE to WDC-10K and WDC-100K. This yields 67 queries that admit feasible solutions for the evaluated configurations, with $k \in \{5, 10, 20\}$, $\alpha \in \{5, 10\}$, and $\tau = 0.1$. Because different (k, α) configurations can produce the same candidate-pool size $\alpha \times k$, we report a single result for identical pool sizes, conservatively using the largest observed cost. Table 1 summarizes the resulting benchmark statistics.

6.3 Effectiveness

We evaluate retrieval effectiveness using precision, recall, and the unionability score of the result set, $U_d(Q, R)$. Precision and recall are computed with respect to the ground-truth unionable tables and therefore measure retrieval quality independently of the distributional constraint. Consequently, a system may achieve high precision and recall while still failing to satisfy the target threshold τ . We report each metric averaged over all queries. To separately assess distributional effectiveness, we report the achieved distributional proportion, $F_{RU\{Q\}}$, also averaged over all queries.

For this evaluation, we use TUS-SMALL, TUS-LARGE, and SANTO-SMALL, for which unionability ground truth is available. STARMIE, STARMIEC (the backend for POST-GREEDY and POST-PREF), and DUTS-2OPTS all use HNSW-based retrieval. For consistency, we use the same HNSW configuration across all systems, setting top- $n = 1000$ to provide a sufficiently large candidate pool and the similarity threshold to $\sigma \geq 0.6$. We set the target distributional threshold to $\tau \geq 0.3$. Table 2 reports the resulting effectiveness.

Table 2: Effectiveness comparison across datasets for $k = 10$, $\alpha = 5$. The highest possible recall@10 is 0.4736, 0.0211, and 0.0249 for SANTO-SMALL, TUS-SMALL, and TUS-LARGE, respectively. All reported values are averaged over all queries, and boldface indicates the best result achieved by a distribution-aware system for each metric.

System	Feasible	Prec.	Recall	$U_d(Q, R)$	$F_{RU\{Q\}}$
SANTO-SMALL					
STARMIE	12/48	0.829	0.4216	115.7	0.269
POST-GREEDY	42/48	0.704	0.3113	93.9	0.338
POST-PREF	35/48	0.612	0.2495	83.3	0.292
DUTS-2OPTS	39/48	0.810	0.3005	95.5	0.360
TUS-SMALL					
STARMIE	5/92	0.886	0.0195	100.1	0.126
POST-GREEDY	43/92	0.429	0.0135	32.8	0.151
POST-PREF	33/92	0.320	0.0116	22.3	0.114
DUTS-2OPTS	84/92	0.864	0.0192	85.1	0.338
TUS-LARGE					
STARMIE	13/142	0.769	0.0191	111.0	0.143
POST-GREEDY	77/142	0.399	0.0129	44.8	0.181
POST-PREF	59/142	0.284	0.0109	28.7	0.141
DUTS-2OPTS	122/142	0.704	0.0184	88.2	0.308

We consider a query *feasible* if the pipeline returns exactly k tables whose union with the query table satisfies the specified distributional constraint on distribution attribute. STARMIE always returns a result set, regardless of whether it satisfies this constraint. In contrast, the distribution-aware methods return an empty result when no feasible set can be identified. For such queries, we assign a value of zero to all reported effectiveness metrics.

The results show that DUTS-2OPTS produces distributionally feasible results consistently more than the alternative methods, while incurring smaller losses in precision, recall, and aggregate unionability relative to the distribution-oblivious STARMIE baseline. Across all three datasets, DUTS-2OPTS fails to return a feasible result for a small number of queries. Further inspection shows that all such failures are attributable to the retrieval stage, which provides an insufficient candidate pool for the subsequent optimization stages. In contrast, some failures of the greedy methods occur even when the retrieved candidate pool contains a feasible solution.

These results therefore establish the effectiveness of DUTS-2OPTS. The remaining question is whether DUTS-2OPTS can maintain practical efficiency as the problem size grows, which we investigate next in our scalability experiments.

6.4 Scalability

Our efficiency analysis focuses on the two main components introduced by DUTS-2OPTS: *fractional programming* in OPTSTAGE 1 and *integer programming* in OPTSTAGE 2. The remaining components, including indexing and unionability computation, build on established techniques from prior work. We report their costs for completeness, but their optimization is beyond the scope of this work.

We investigate whether Dinkelbach’s well-known rapid convergence also holds at data-lake scale. In particular, we measure

the number of iterations required for convergence across queries and dataset sizes, in addition to its time cost.

For the integer programming stage, we evaluate the cost of solving the resulting binary optimization problem and study how this cost scales with the size of the candidate set. Together, these experiments quantify the computational overhead of the optimization machinery introduced by DUTS-2OPTS and assess whether it remains practical as the search space grows.

6.4.1 Dinkelbach Convergence. We evaluate the convergence of Dinkelbach’s algorithm on both SANTOS-LARGE and the WDC tiers. We arbitrary fix $k = 10$ and $\alpha = 3$ and vary top- n for HNSW retrieval, in the range of $\{5000, 10000, \dots, 30000\}$, inducing different input sizes, the induced input sizes are reported in Appendix Table 4. On SANTOS-LARGE the algorithm converges in at most 2.3 iterations on average, with mean execution time below 0.3 ms across all input sizes. We observe similarly fast convergence on the WDC tiers: even for inputs of approximately 5000 candidates, convergence requires fewer than three iterations on average and less than 12 ms. These results indicate that Dinkelbach’s algorithm converges rapidly across datasets and input sizes and contributes only a small overhead to the pipeline. Detailed results are provided in Appendix Table 6 and Figure 5.

6.4.2 ILP. We evaluate the computational cost of OPTSTAGE 2 by separately measuring the LP pre-check and ILP solver. By construction, all evaluated queries admit a feasible solution, ensuring that the complete OPTSTAGE 2 pipeline is executed for every query. On SANTOS-LARGE, across all 46 queries, the combined mean execution time of these two components remains below 2.4 ms across all evaluated input sizes. The LP pre-check is consistently faster than solving the full ILP; however, it is not decisive for any of these queries and therefore does not eliminate any ILP invocations.

The WDC tiers provide a complementary setting. While query selection for the WDC-10K and WDC-100K tiers was designed to ensure the existence of feasible answers, the WDC-1M tier provides a less constrained evaluation. On this tier, the LP pre-check is decisive for 7 of the 67 queries, avoiding unnecessary ILP invocations for these cases. Despite the larger input sizes, the combined mean runtime of the LP pre-check and ILP solver remains below 6 ms across all evaluated configurations. Overall, these results indicate that OPTSTAGE 2 incurs only a small computational overhead, while the LP pre-check can additionally reduce this cost when infeasible instances arise. Detailed results for the WDC experiments are reported in Appendix Table 7.

6.4.3 End-to-End. We next evaluate the runtime breakdown of the complete pipeline. Figure 4 reports the mean execution time of each pipeline component of DUTS-2OPTS, together with the end-to-end latency, averaged over all queries that execute the complete pipeline.

Probe captures all operations preceding OPTSTAGE 1, including probing HNSW and inverted indexes, intersecting their results, and retrieving the corresponding synopses. OPTSTAGE 1 incurs a small and relatively stable overhead across all datasets, exhibiting only minor sensitivity to the candidate-pool size. In contrast, unionability scoring scales approximately linearly with the candidate-pool size, as expected, since each candidate in the pool must be scored; importantly, its cost is independent of the overall corpus size once the candidate pool has been retrieved. OPTSTAGE 2 remains comparatively inexpensive, increasing only modestly as the candidate pool grows. Overall, the end-to-end

Table 3: Index construction time and storage across datasets.

Dataset	Data Size	HNSW		Inverted Index	
		Build (s)	Size (MB)	Build (s)	Size (MB)
SANTOS-LARGE	4.9 GB	224.7	281.8	109.7	16.9
WDC-10K	45 MB	13.9	163.6	2.7	8.75
WDC-100K	441 MB	200.8	1,633.0	23.4	76.6
WDC-1M	4.4 GB	1,284.8	16,322.8	49.6	670.5

latency, from receiving a query to producing the final result, remains approximately 400 ms or less across all evaluated configurations.

A notable exception to the otherwise stable probing cost is WDC-1M, where the probe time increases substantially. This increase does not originate from the HNSW probe, whose runtime remains nearly constant across WDC-10K, WDC-100K, and WDC-1M ($8.1 \rightarrow 9.3 \rightarrow 10.2$ ms), despite a $100\times$ increase in the number of tables. Instead, the increase is driven by the inverted-index overlap probe. Although locating the posting list for a distributional value is inexpensive, the probe must enumerate and materialize all matching (table, attribute) pairs, making its cost output-sensitive. Across the three WDC tiers, the number of such matches grows by approximately an order of magnitude per tier ($3.3K \rightarrow 34K \rightarrow 341K$), while the corresponding probe time increases from 1.2 to 15.5 and 203.5 ms. Thus, the observed increase is driven primarily by the growing posting-list length rather than the corpus size itself. At the WDC-1M scale, the overlap probe accounts for approximately 84% of the retrieval cost and is therefore the primary source of the increased end-to-end latency.

6.4.4 Index Cost. Table 3 reports the construction time and storage cost of both indexes. As expected, both metrics increase with the size of the WDC tiers. For SANTOS-LARGE, the HNSW index contains 84,157 elements, while the inverted index contains 277,707 distinct values and 887,428 (table, attribute) postings. HNSW construction time is primarily determined by graph construction, distance computations, and layer assignment, rather than solely by the number of indexed elements. In contrast, inverted-index construction requires scanning the data-lake tables and materializing the corresponding value postings.

Interestingly, although SANTOS-LARGE and WDC-1M have similar on-disk dataset sizes, their index sizes and construction times differ substantially. This difference arises because WDC-1M contains more categorical attributes and distinct values than SANTOS-LARGE, resulting in a larger number of elements and postings to index. Thus, raw dataset size alone does not determine indexing cost; the number and cardinality of the indexed attributes also play an important role.

6.5 Ablation Study: Benefit of OPTSTAGE 1

In principle, OPTSTAGE 1 can be bypassed by solving the ILP directly over the entire retrieved candidate set. We evaluate the benefit of our two-stage design by comparing its runtime, measured from immediately before OPTSTAGE 1 to completion, with a variant that skips OPTSTAGE 1 and directly invokes the ILP. We set top- n to 30,000 to obtain large candidate pools and use $k = 10$ and $\alpha = 3$, arbitrarily selected from the feasible configurations. All queries complete the full pipeline under these settings except

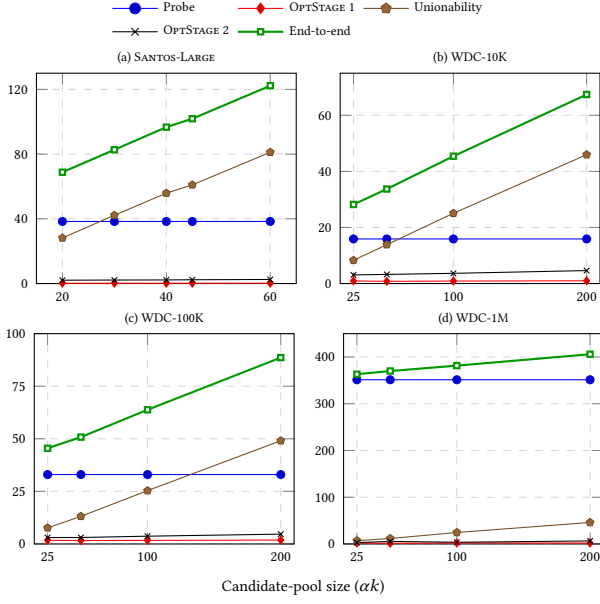


Figure 4: Mean per-step runtime (ms) as the candidate-pool size ak increases.

for one query on WDC-1M. Overall, OPTSTAGE 1 substantially reduces the candidate set passed to the ILP, lowering optimization cost and improving end-to-end runtime. Moreover, across the WDC tiers, the speedup increases with input size, highlighting the growing benefit of OPTSTAGE 1 as the candidate pool scales. Detailed results are provided in the Appendix, Table 5.

7 Related Work

Table Discovery. Given a query table, discovering useful tables for expansion has been extensively studied, with joinable and unionable table search receiving particular attention [12]. For unionable search, Nargesian et al. [24] define table unionability through attribute-level compatibility, estimated using values, ontologies, and semantic embeddings. Subsequent systems incorporate additional signals and develop richer semantic representations of attributes [4, 13, 15, 17]. Recent work augments unionability with the novelty contributed by retrieved tables at the tuple [18] or table level [16]. In contrast, we consider an orthogonal requirement: retrieving highly unionable tables whose union with the query satisfies a user-specified distributional requirement.

Distribution-Aware Dataset Discovery. Prior work has incorporated distributional requirements into data acquisition and integration. Nargesian et al. [23] introduce *Data Distribution Tailoring* (DT), which constructs a dataset satisfying user-specified distribution requirements by acquiring tuples from multiple data sources while minimizing acquisition cost. They consider both settings where source distributions are known and settings where they must be learned during acquisition. Subsequent work further develops cost-efficient distribution tailoring [7]. Asudeh and Nargesian [1] consider distribution-aware data preparation in data markets, integrating data discovery, data integration, distribution requirements, and acquisition cost into a unified query-answering framework. These works share with DUTS the goal of incorporating distributional requirements into data preparation, but address a different problem. They assume a target schema

and acquire tuples from available sources, optimizing acquisition cost. In contrast, DUTS assumes no predefined schema: it starts from a query table and searches a data lake for a fixed-size set of unionable tables that maximizes unionability while satisfying a user-specified distributional requirement. Thus, DUTS operates at the table rather than tuple level, targeting settings where tuples cannot be acquired independently and entire tables must be selected.

More recently, Behme et al. [2] propose *Fainder*, an index for distribution-aware dataset search over decentralized repositories. Fainder combines keyword search with percentile predicates over histogram summaries to retrieve individual datasets satisfying specified distributional requirements. In contrast, DUTS starts from a *query table* and retrieves a fixed-size set of unionable tables, jointly considering unionability and a distributional requirement. Crucially, DUTS evaluates the requirement over the union of the query and selected tables, rather than independently over each candidate. Thus, a table need not individually satisfy the target distribution to contribute to a feasible result. DUTS therefore brings distribution awareness to *unionable table search*, where both table unionability and interactions among selected tables determine the result.

Fractional and Linear Programming. We formulate one stage of DUTS-2OPTS as a fractional program and solve it using Dinkelbach’s method [11]. Fractional programming has been applied to optimization problems involving ratio-based objectives and constraints. For example, Güngör et al. [14] formulate a task-assignment problem, in which tasks are uniquely assigned to individuals while balancing workload, as a constrained fractional 0–1 program. Fractional formulations have also been studied for incorporating fairness criteria into optimization models [9]. Related optimization formulations appear in fair ranking. Singh and Joachims [26] maximize expected utility under linear fairness constraints, while Biega et al. [3] formulate attention allocation as an integer linear program to mitigate position bias. These formulations share with DUTS-2OPTS the general structure of optimizing utility subject to distributional constraints. However, DUTS-2OPTS considers a different setting: selecting a fixed-size set of unionable tables while enforcing a target distribution over their resulting union.

8 Conclusion and Future Work

We tackled the problem of identifying a set of unionable tables from a data lake that jointly satisfies a distributional requirement over a query’s categorical attribute. Since the constraint is evaluated over the union rather than individual tables, the problem does not decompose into independent per-table decisions. We proved this problem to be NP-hard. We studied two paradigms for addressing this problem. The *unionability-first* paradigm first retrieves highly unionable tables using an existing discovery system, such as STARMIE, and then repairs the result through table swaps. We showed that finding the minimum-cost repair is itself NP-complete and developed a greedy swap heuristic. In contrast, our *distribution-first* paradigm, DUTS-2OPTS, integrates the distributional requirement into retrieval through a two-stage solution. Stage1 solves a 0–1 linear-fractional program to construct a distribution-optimal pool of ak candidates. Stage2 then selects the final k tables by maximizing unionability over this pool subject to the distributional constraint, formulated as a 0–1 linear program. This design limits Stage 2 to at most ak candidates, independent of the data-lake size.

Through extensive evaluation, we show that the distribution-first paradigm is more effective than the unionability-first alternative. Across multiple benchmarks, DUTS-2OptS returns distributionally feasible results for the large majority of queries while largely preserving the precision, recall, and unionability of the distribution-oblivious baseline. In contrast, the swap heuristics either satisfy the distributional constraints for substantially fewer queries or incur a considerably larger loss in unionability when doing so. The two paradigms also exhibit fundamentally different failure modes. All observed DUTS-2OptS failures arise when retrieval produces a candidate pool containing no feasible solution; whenever a feasible solution is present in the pool, the optimization identifies it. In contrast, the heuristics can fail even when the retrieved pool contains a feasible solution, because greedy ordering or filtering may eliminate candidates required to construct it. Despite its optimization-based design, DUTS-2OptS remains efficient at scale. End-to-end latency stays at approximately 400 ms or below on a one-million-table data lake, with runtime dominated not by optimization but by the unionability computation and the inverted-index probe, whose cost is output-sensitive to the posting-list length.

Future work. Our formulation considers a single distributional requirement over one value set of one attribute. A natural extension is to support requirements over multiple attributes and, importantly, their dependencies, enabling richer notions of representation such as intersectional constraints. Supporting full target distributions over $\text{Dom}(d)$ is another direction; both extensions change the structure of the distributional constraint and require corresponding modifications to the optimization formulations in both stages.

Our results also identify retrieval, rather than optimization, as the primary bottleneck in both efficiency and solution quality. At the one-million-table tier, the overlap probe accounts for approximately 84% of retrieval time, suggesting that more efficient index structures could provide immediate performance gains. A more fundamental direction is to make retrieval itself distribution-aware, so that the index produces candidates suitable for satisfying the distributional requirement rather than filtering candidates afterward. In particular, designing an index that efficiently supports query-specific distributional constraints whose feasibility depends jointly on a set of retrieved items remains an interesting open problem.

References

- [1] Abolfazl Asudeh and Fatemeh Nargesian. 2022. Towards Distribution-aware Query Answering in Data Markets. *Proc. VLDB Endow.* 15, 11 (2022), 3137–3144. doi:10.14778/3551793.3551858
- [2] Lennart Behme, Sainyam Ghalotra, Kaustubh Beedkar, and Volker Markl. 2024. Fainder: A Fast and Accurate Index for Distribution-Aware Dataset Search. *Proc. VLDB Endow.* 17, 11 (2024), 3269–3282. doi:10.14778/3681954.3681999
- [3] Asia J. Biega, Krishna P. Gummadi, and Gerhard Weikum. 2018. Equity of Attention: Amortizing Individual Fairness in Rankings. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, SIGIR 2018, Ann Arbor, MI, USA, July 08-12, 2018*, Kevyn Collins-Thompson, Qiaozhu Mei, Brian D. Davison, Yiqun Liu, and Emine Yilmaz (Eds.). ACM, 405–414. doi:10.1145/3209978.3210063
- [4] Alex Bogatu, Alvaro A. A. Fernandes, Norman W. Paton, and Nikolaos Konstantinou. 2020. Dataset Discovery in Data Lakes. In *ICDE*. 709–720.
- [5] Alberto Caprara, Hans Kellerer, Ulrich Pferschy, and David Pisinger. 2000. Approximation algorithms for knapsack problems with cardinality constraints. *Eur. J. Oper. Res.* 123, 2 (2000), 333–345. doi:10.1016/S0377-2217(99)00261-1
- [6] L. Elisa Celis, Damian Straszak, and Nisheeth K. Vishnoi. 2018. Ranking with Fairness Constraints. In *45th International Colloquium on Automata, Languages, and Programming, ICALP 2018, Prague, Czech Republic, July 9-13, 2018 (LIPIcs)*, Ioannis Chatzigiannakis, Christos Kaklamanis, Daniel Marx, and Donald Sannella (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 28:1–28:15. doi:10.4230/LIPIcs.ICALP.2018.28
- [7] Jiwon Chang, Bohan Cui, Fatemeh Nargesian, Abolfazl Asudeh, and H. V. Jagadish. 2024. Data distribution tailoring revisited: cost-efficient integration of representative data. *VLDB J.* 33, 5 (2024), 1283–1306. doi:10.1007/S00778-024-00849-W
- [8] Abraham Charnes and William W Cooper. 1962. Programming with linear fractional functionals. *Naval Research logistics quarterly* 9, 3-4 (1962), 181–186.
- [9] Violet Xinying Chen and JN Hooker. 2021. A guide to formulating equity and fairness in an optimization model. *Preprint* (2021), 162–174.
- [10] Jan Chomicki. 2003. Preference formulas in relational queries. *ACM Trans. Database Syst.* 28, 4 (2003), 427–466. doi:10.1145/958942.958946
- [11] Werner Dinkelbach. 1967. On nonlinear fractional programming. *Management science* 13, 7 (1967), 492–498.
- [12] Grace Fan, Jin Wang, Yuliang Li, and Renée J. Miller. 2023. Table Discovery in Data Lakes: State-of-the-art and Future Directions. In *SIGMOD*, Sudipto Das, Ippokratis Pandis, K. Selçuk Candan, and Sihem Amer-Yahia (Eds.). ACM, 69–75. doi:10.1145/3555041.3589409
- [13] Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée J. Miller. 2023. Semantics-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning. *Proc. VLDB Endow.* 16, 7 (2023), 1726–1739. doi:10.14778/3587136.3587146
- [14] Murat Güngör. 2019. A fractional 0–1 program for task assignment with respect to preferences. *Computers & Industrial Engineering* 131 (2019), 263–268. doi:10.1016/j.cie.2019.03.048
- [15] Xuming Hu, Shen Wang, Xiao Qn, Chuan Lei, Zhengyuan Shen, Christos Faloutsos, Asterios Katsifodimos, George Karypis, Lijie Wen, and Philip S. Yu. 2023. Automatic Table Union Search with Tabular Representation Learning. In *Findings of the Association for Computational Linguistics: ACL 2023, Toronto, Canada, July 9-14, 2023*, Anna Rogers, Jordan L. Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, 3786–3800. doi:10.18653/V1/2023.FINDINGS-ACL.233
- [16] Besat Kassaie and Renée J. Miller. 2026. Novel Table Search. In *42nd IEEE International Conference on Data Engineering, ICDE 2026, Montreal, QC, Canada, May 4-8, 2026*. IEEE, 1981–1993. doi:10.1109/ICDE65706.2026.00150
- [17] Aamod Khatiwada, Grace Fan, Roe Shraga, Zixuan Chen, Wolfgang Gatterbauer, Renée J. Miller, and Mirek Riedewald. 2023. SANTOS: Relationship-based Semantic Table Union Search. In *SIGMOD*.
- [18] Aamod Khatiwada, Roe Shraga, and Renée J. Miller. 2026. Diverse Unionable Tuple Search: Novelty-Driven Discovery in Data Lakes. In *Proceedings 29th International Conference on Extending Database Technology, EDBT 2026, Tampere, Finland, March 24-27, 2026*, Wolfgang Lehner, Vanessa Braganholo, Kostas Stefanidis, Zheyang Zhang, Alexander Krause, and João Felipe Nicolaci Pimentel (Eds.). OpenProceedings.org, 42–55. doi:10.48786/EDBT.2026.04
- [19] Oliver Lehmberg, Dominique Ritze, Robert Meusel, and Christian Bizer. 2016. A Large Public Corpus of Web Tables containing Time and Context Metadata. In *WWW (Companion Volume)*. ACM, 75–76.
- [20] Yuri A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.
- [21] Tomomi Matsui, Yasufumi Saruwatari, and Maiko Shigeno. 1992. An analysis of dinkelbach’s algorithm for 0-1 fractional programming problems. *METR92-14* (1992).
- [22] Sameer Mehta, Milind Dawande, Ganesh Janakiraman, and Vijay S. Mookerjee. 2021. How to Sell a Data Set? Pricing Policies for Data Monetization. *Inf. Syst. Res.* 32, 4 (2021), 1281–1297. doi:10.1287/ISRE.2021.1027
- [23] Fatemeh Nargesian, Abolfazl Asudeh, and H. V. Jagadish. 2021. Tailoring Data Source Distributions for Fairness-aware Data Integration. *Proc. VLDB Endow.* 14, 11 (2021), 2519–2532. doi:10.14778/3476249.3476299
- [24] Fatemeh Nargesian, Erkang Zhu, Ken Q. Pu, and Renée J. Miller. 2018. Table Union Search on Open Data. *Proc. VLDB Endow.* 11, 7 (2018), 813–825. doi:10.14778/3192965.3192973
- [25] Bruno Scarone, Alfredo Viola, and Renée J. Miller. 2026. Data Bias Mitigation under Coverage Constraints & The Price of Fairness. In *Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency, FAccT 2026, Montreal, QC, Canada, June 25-28, 2026*. ACM, 4935–4968. doi:10.1145/3805689.3812359
- [26] Ashudeep Singh and Thorsten Joachims. 2018. Fairness of Exposure in Rankings. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19-23, 2018*, Yike Guo and Faisal Ferooq (Eds.). ACM, 2219–2228. doi:10.1145/3219819.3220088
- [27] Fengqi You, Pedro M. Castro, and Ignacio E. Grossmann. 2009. Dinkelbach’s algorithm as an efficient method to solve a class of MINLP models for large-scale cyclic scheduling problems. *Comput. Chem. Eng.* 33, 11 (2009), 1879–1889. doi:10.1016/J.COMPCHEMENG.2009.05.014
- [28] Jiayao Zhang, Yuran Bi, Mengye Cheng, Jinfei Liu, Kui Ren, Qiheng Sun, Yihang Wu, Yang Cao, Raul Castro Fernandez, Haifeng Xu, Ruoxi Jia, Yongchan Kwon, Jian Pei, Jiachen T. Wang, Haocheng Xia, Li Xiong, Xiaohui Yu, and James Zou. 2024. A Survey on Data Markets. *CoRR abs/2411.07267* (2024). doi:10.48550/ARXIV.2411.07267 arXiv:2411.07267
- [29] Chen Zhao, Kai Jiang, Xintao Wu, Haoliang Wang, Latifur Khan, Christian Grant, and Feng Chen. 2024. Algorithmic Fairness Generalization under Covariate and Dependence Shifts Simultaneously. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25-29, 2024*, Ricardo Baeza-Yates and Francesco Bonchi (Eds.). ACM, 4419–4430. doi:10.1145/3637528.3671909

1513	[30] Erkang Zhu, Dong Deng, Fatemeh Nargesian, and Renée J. Miller. 2019. JOSIE: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes. In <i>Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019</i> ,	Peter A. Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska (Eds.). ACM, 847–864. doi:10.1145/3299869.3300065	1576
1514			1577
1515			1578
1516			1579
1517			1580
1518			1581
1519			1582
1520			1583
1521			1584
1522			1585
1523			1586
1524			1587
1525			1588
1526			1589
1527			1590
1528			1591
1529			1592
1530			1593
1531			1594
1532			1595
1533			1596
1534			1597
1535			1598
1536			1599
1537			1600
1538			1601
1539			1602
1540			1603
1541			1604
1542			1605
1543			1606
1544			1607
1545			1608
1546			1609
1547			1610
1548			1611
1549			1612
1550			1613
1551			1614
1552			1615
1553			1616
1554			1617
1555			1618
1556			1619
1557			1620
1558			1621
1559			1622
1560			1623
1561			1624
1562			1625
1563			1626
1564			1627
1565			1628
1566			1629
1567			1630
1568			1631
1569			1632
1570			1633
1571			1634
1572			1635
1573			1636
1574			1637
1575			1638

A Additional Material

This appendix provides additional experimental results omitted from the main paper due to space constraints.

A.1 Proofs

Theorem 3.1.

PROOF SKETCH. We reduce from the Exact k -Item Knapsack Problem (E- k KP), which is known to be NP-hard [5]. An instance of E- k KP consists of items with profits p_j , weights w_j , a capacity c , a required cardinality k . The objective is to select exactly k items with total weight at most c while maximizing total profit.

For each item j , we construct a table T_j containing exactly c tuples, of which $c - w_j$ contain values from $\mathcal{V}$ and the remaining w_j do not. Consequently,

$$n_j = c, \quad N_j = c - w_j, \quad u_j = p_j.$$

Thus, all candidate tables have the same size, while heavier items correspond to fewer tuples matching $\mathcal{V}$. We further construct a fixed query table Q containing one tuple whose value belongs to $\mathcal{V}$. Hence, $n_Q = N_Q = 1$. Consider any subset S of exactly k candidate tables. The achieved distribution over the union of Q and the selected tables is

$$\begin{aligned} F_S &= \frac{N_Q + \sum_{T_j \in S} N_j}{n_Q + \sum_{T_j \in S} n_j} \\ &= \frac{1 + \sum_{T_j \in S} (c - w_j)}{1 + kc} = \frac{1 + kc - \sum_{T_j \in S} w_j}{1 + kc}. \end{aligned}$$

Choose

$$\tau = \frac{1 + (k-1)c}{1 + kc}.$$

Then the distributional constraint becomes

$$\begin{aligned} F_S \geq \tau &\iff \frac{1 + kc - \sum_{T_j \in S} w_j}{1 + kc} \geq \frac{1 + (k-1)c}{1 + kc} \\ &\iff \sum_{T_j \in S} w_j \leq c, \end{aligned}$$

which is exactly the knapsack capacity constraint.

Moreover, the unionability objective satisfies

$$U_d(Q, S) = \sum_{T_j \in S} u_j = \sum_{T_j \in S} p_j.$$

Therefore, maximizing unionability is equivalent to maximizing the total profit.

Consequently, feasible selections of exactly k tables correspond to feasible selections of exactly k items, preserving both the capacity constraint and the objective. Thus, an optimal solution to Problem 1 yields an optimal solution to the corresponding E- k KP instance. Since the reduction is computable in polynomial time and E- k KP is NP-hard, Problem 1 is NP-hard. $\square$

Claim 1.

PROOF. We wish to show that

$$F_R = \frac{\sum_{T_i \in R} N_i}{\sum_{T_i \in R} n_i} < \tau^* \implies \exists T_j \in R : \frac{N_j}{n_j} < \tau^*.$$

Assume by contradiction that every table satisfies the distributional constraint individually:

$$\forall T_i \in R : \frac{N_i}{n_i} \geq \tau^* \implies N_i \geq \tau^* n_i.$$

Summing over all tables in R yields

$$\sum_{T_i \in R} N_i \geq \tau^* \sum_{T_i \in R} n_i,$$

which gives $F_R \geq \tau^*$, contradicting the assumption $F_R < \tau^*$. $\square$

Theorem 5.1.

PROOF. We reduce from the NP-complete problem SUBSET SUM. Given positive integers $a_1, \dots, a_n$ and a target W satisfying

$$0 < W < \sum_{i=1}^n a_i,$$

SUBSET SUM asks whether there exists a subset $I \subseteq \{1, \dots, n\}$ such that

$$\sum_{i \in I} a_i = W.$$

Let

$$M = \sum_{i=1}^n a_i, \quad K = M + 1.$$

Thus, $0 < a_i < K$ for every i . The value M is used to give all constructed result and candidate tables the same cardinality, ensuring that a repair does not change their contribution to the denominator of the distributional ratio. The value K ensures that the candidate unionability scores defined below lie strictly between 0 and 1, while making the unionability loss associated with candidate c_i proportional to a_i .

For each item a_i , construct one result table $t_i \in R$ and one candidate table $c_i \in C$ with statistics

$$N(t_i) = 0, \quad n(t_i) = M, \quad U(Q, t_i) = 1,$$

and

$$N(c_i) = a_i, \quad n(c_i) = M, \quad U(Q, c_i) = \frac{K - a_i}{K}.$$

Thus, selecting candidate c_i contributes exactly a_i tuples in $\mathcal{V}$ to the repaired result. At the same time, replacing a result table of unionability 1 by c_i incurs a unionability loss of

$$1 - \frac{K - a_i}{K} = \frac{a_i}{K}.$$

Hence, each SUBSET SUM item a_i is encoded both in its contribution to the distributional constraint and, up to the constant factor $1/K$, in its repair cost. Since $0 < a_i < K$,

$$0 < U(Q, c_i) < 1 = U(Q, t_i),$$

so every candidate satisfies

$$U(Q, c_i) < U(Q, t_i).$$

Moreover, since $0 < a_i \leq M$, every constructed table satisfies

$$0 \leq N(\cdot) \leq n(\cdot),$$

and is therefore valid.

We construct the query table Q with one tuple whose value for d is not in $\mathcal{V}$. Thus,

$$N_Q = 0, \quad n_Q = 1.$$

Let

$$R = \{t_1, \dots, t_n\}, \quad k = n,$$

and choose

$$\tau = \frac{W}{nM + 1}.$$

Since

$$0 < W < M < nM + 1,$$

we have $\tau \in (0, 1)$. Furthermore,

$$F_Q = \frac{N_Q}{n_Q} = 0 < \tau.$$

Therefore, under the definition of the swappable set for the case $F_Q < \tau$, all result tables are eligible for replacement, and hence

$$R_{\text{sw}} = R = \{t_1, \dots, t_n\}.$$

The initial result is distributionally infeasible, since

$$F(\mathbb{T}(R \cup \{Q\})) = \frac{N_Q + \sum_{i=1}^n N(t_i)}{n_Q + \sum_{i=1}^n n(t_i)} = \frac{0}{nM + 1} = 0 < \tau.$$

Thus, the constructed instance requires a repair. The threshold is chosen so that satisfying the distributional constraint after repair corresponds exactly to selecting candidates whose associated SUBSET SUM values sum to at least W .

A repair selects a subset of candidates $B \subseteq C$ and replaces an equally sized subset $A \subseteq R_{\text{sw}}$, yielding

$$R' = (R \setminus A) \cup B, \quad |A| = |B|.$$

Since all result tables have identical statistics,

$$N(t_i) = 0, \quad n(t_i) = M, \quad U(Q, t_i) = 1,$$

the distributional score and repair cost depend only on the selected candidate set B and not on the particular choice of A .

Because every removed and inserted table contains exactly M tuples, the repair preserves the cardinality of the result:

$$n(R') = nM.$$

Including the query table, the total cardinality is therefore

$$n(R' \cup \{Q\}) = nM + 1.$$

The remaining result tables and Q contribute no tuples in $\mathcal{V}$, whereas each selected candidate c_i contributes a_i such tuples. Hence,

$$F(\mathbb{T}(R' \cup \{Q\})) = \frac{\sum_{c_i \in B} a_i}{nM + 1}.$$

By the definition of τ ,

$$\begin{aligned} F(\mathbb{T}(R' \cup \{Q\})) \geq \tau &\iff \frac{\sum_{c_i \in B} a_i}{nM + 1} \geq \frac{W}{nM + 1} \\ &\iff \sum_{c_i \in B} a_i \geq W. \end{aligned} \quad (6)$$

Thus, Equation (6) rewrites the feasibility constraint: a repair is distributionally feasible if and only if the SUBSET SUM values associated with its selected candidates sum to at least W . We next rewrite the repair objective. Recall that the repair cost is

$$L(A, B) = U_d(Q, R) - U_d(Q, R'),$$

where

$$U_d(Q, S) = \sum_{T \in S} U_d(Q, T).$$

Since

$$R' = (R \setminus A) \cup B,$$

the unionability scores of all unchanged tables cancel, leaving

$$L(A, B) = \sum_{t_i \in A} U(Q, t_i) - \sum_{c_i \in B} U(Q, c_i).$$

Using $|A| = |B|$ and substituting the constructed unionability values,

$$\begin{aligned} L(A, B) &= |B| - \sum_{c_i \in B} \frac{K - a_i}{K} \\ &= \sum_{c_i \in B} \left(1 - \frac{K - a_i}{K}\right) \\ &= \frac{1}{K} \sum_{c_i \in B} a_i. \end{aligned} \quad (7)$$

Thus, minimizing the repair cost is equivalent, up to the positive constant factor $1/K$, to minimizing the sum of the SUBSET SUM values associated with the selected candidates.

Combining Equations (6) and (7), computing a minimum-cost feasible repair for the constructed instance is equivalent to solving

$$\begin{aligned} \min_{B \subseteq C} \quad & \frac{1}{K} \sum_{c_i \in B} a_i \\ \text{s.t.} \quad & \sum_{c_i \in B} a_i \geq W. \end{aligned} \quad (8)$$

That is, among all subsets whose associated values sum to at least W , the repair problem selects one having the smallest such sum.

Let L^* denote the minimum repair cost. Since every feasible repair satisfies

$$\sum_{c_i \in B} a_i \geq W,$$

Equation (7) implies

$$L^* \geq \frac{W}{K}. \quad (9)$$

Equality holds precisely when there exists a feasible candidate set whose associated values sum exactly to W . Therefore,

$$L^* = \frac{W}{K} \iff \exists B \subseteq C : \sum_{c_i \in B} a_i = W. \quad (10)$$

To see both directions explicitly, suppose first that the original SUBSET SUM instance is a yes-instance. Then there exists $I \subseteq \{1, \dots, n\}$ such that

$$\sum_{i \in I} a_i = W.$$

Selecting the corresponding candidates

$$B = \{c_i \mid i \in I\}$$

gives, by Equation (6), a feasible repair and, by Equation (7), repair cost

$$L(A, B) = \frac{W}{K}.$$

Since Equation (9) shows that no feasible repair can have smaller cost, we obtain

$$L^* = \frac{W}{K}.$$

Conversely, suppose that

$$L^* = \frac{W}{K}.$$

Let B^* be an optimal repair. By Equation (7),

$$\frac{1}{K} \sum_{c_i \in B^*} a_i = \frac{W}{K},$$

and therefore

$$\sum_{c_i \in B^*} a_i = W.$$

The indices of the candidates in B^* therefore form a subset whose corresponding a_i values sum exactly to W , so the original SUBSET SUM instance is a yes-instance. Hence,

$$\text{SUBSET SUM IS A YES-INSTANCE} \iff L^* = \frac{W}{K}.$$

Consequently, if a minimum-cost feasible repair could be computed in polynomial time, we could decide SUBSET SUM in polynomial time by constructing the corresponding repair instance, computing its optimal cost L^* , and testing whether $L^* = W/K$. The key observation is that feasibility requires the selected values to sum to at least W , while the repair objective minimizes this same sum. Consequently, the minimum possible repair cost is W/K , and this cost is attained if and only if some subset sums

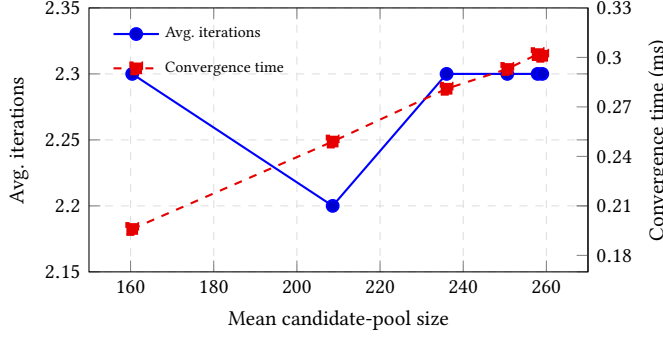


Figure 5: Convergence behavior of Dinkelbach's algorithm with increasing candidate-pool size on SANTOS-LARGE, averaged over 46 query tables. The solid line reports the average number of iterations (left axis), while the dashed line reports the average convergence time (right axis).

exactly to W . Finally, the construction operates only on the statistics required by the repair problem, namely $N(T)$, $n(T)$, and $U(Q, T)$. It creates $2n$ table-statistic records together with the constant-size query table Q , and all numerical quantities involved are representable using a number of bits polynomial in the size of the SUBSET SUM instance. Hence, the reduction is computable in polynomial time.

The reduction establishes hardness even for the restricted case in which $F_Q < \tau$, all original result tables contain zero tuples in $\mathcal{V}$, all result and candidate tables have equal cardinality, and all result tables are swappable. Since SUBSET SUM is NP-complete, computing a minimum-cost feasible repair is therefore NP-hard. $\square$

Proposition 5.2.

PROOF. Soundness. The algorithm returns a non-empty result only after certifying $F_{R \cup \{Q\}} \geq \tau$ (Line 13); it never returns a distributionally infeasible R . Every committed swap strictly increases $F(\mathbb{T}(R \cup Q))$ by construction (Line 11, 12), so the sequence of committed swaps is monotonically improving.

Complexity. The outer loop iterates over at most $|R_{sw}|$ violating tables. **FINDBEST** evaluates each of the (at most) $|C|$ remaining candidates in $O(1)$ time given precomputed N_i and n_i values, so each call costs $O(|C|)$. The overall runtime is therefore $O(|R_{sw}| \cdot |C|)$. $\square$

Claim 2.

PROOF. For $t \in R_{sw}$, let $N_0^{(t)}$ be the total number of tuples in $\mathcal{V}$ contributed by all tables in $R \setminus \{t\}$, let $W_0^{(t)} = \sum_{T_i \in R \setminus \{t\}} (n_i - N_i)$ be the total number of tuples *not* in $\mathcal{V}$ contributed by the same tables, and let $B_t = (N_0^{(t)}, W_0^{(t)})$. Let $c_1, c_2 \in C$ be candidate tables with $N(c_1) \neq N(c_2)$ or $n(c_1) - N(c_1) \neq n(c_2) - N(c_2)$.

Let

$$D(B_t) = F(\mathbb{T}(R(t, c_1))) - F(\mathbb{T}(R(t, c_2))).$$

Substituting and simplifying yields

$$D(B_t) = \frac{A \times N_0^{(t)} + B \times W_0^{(t)} + C}{(N_0^{(t)} + W_0^{(t)} + n(c_1)) \times (N_0^{(t)} + W_0^{(t)} + n(c_2))}$$

for constants $A = (n(c_2) - N(c_2)) - (n(c_1) - N(c_1))$, $B = N(c_1) - N(c_2)$, and $C = N(c_1)(n(c_2) - N(c_2)) - N(c_2)(n(c_1) - N(c_1))$.

At least one of A or B is nonzero. The denominator is positive, and the numerator is a linear function of $N_0^{(t)}$ and $W_0^{(t)}$ plus a constant. Different violating tables induce different $N_0^{(t)}$ and $W_0^{(t)}$; there exists a threshold line in the $(N_0^{(t)}, W_0^{(t)})$ -plane where $D(B_t) = 0$. Choosing two violating tables t_1, t_2 whose induced $(N_0^{(t_1)}, W_0^{(t_1)})$ and $(N_0^{(t_2)}, W_0^{(t_2)})$ lie on opposite sides of this line yields $D(B_{t_1}) > 0$ and $D(B_{t_2}) < 0$, hence

$$F(\mathbb{T}(R(t_1, c_1))) > F(\mathbb{T}(R(t_1, c_2))) \quad \text{and}$$

$$F(\mathbb{T}(R(t_2, c_1))) < F(\mathbb{T}(R(t_2, c_2))).$$

Therefore the preferred replacement candidate depends on the violating table. $\square$

Claim 3.

PROOF. Adopting the notation of Claim 2, we consider each condition in turn. The first, second, and third conditions each imply $D(B_t) > 0$ regardless of the choice of t . The fourth condition implies $D(B_t) = 0$. $\square$

A.2 STARMIE Settings

We fine-tuned the STARMIE model on the original version of the SANTO-SMALL dataset [17] and used the resulting model to generate 768-dimensional embeddings for all attributes across our datasets. We chose SANTO-SMALL as the fine-tuning dataset arbitrarily and applied the resulting model to all datasets without further dataset-specific tuning.

We used the following hyperparameters for fine-tuning:

batch size = 64, learning rate = $5e - 5$, max. seq. length = 128, proj. dim. = 768, augment_op = drop_col, sample_meth = tfidf_entity, table_order=column and num. of epochs=3. Refer to the official STARMIE codebase for hyperparameter details.⁶

A.3 Parameter Selection for SANTOS Large

For the scalability experiments, we adopt a parameter-selection strategy designed to retain sufficiently large candidate pools throughout the pipeline. The random selection of distribution attribute and target value used in our effectiveness experiments yields at least 50 candidates for only 19 of the 78 eligible SANTOS-LARGE queries, making it unsuitable for stress-testing the system. We therefore select distribution attribute and target value to maximize the number of candidates entering the distribution-aware candidate-selection stage (Section 4.3).

Selecting distribution attribute and target value. For each of the 78 eligible query tables, we consider every pair (A, v) , where A is a categorical query attribute and v is a distinct value occurring in A , as a potential (distribution attribute, target value) assignment. We use the existing inverted index to estimate the candidate-pool size of each assignment by counting the table-attribute pairs containing v . Since this estimate does not account for semantic attribute compatibility, we retain the 20 assignments with the largest estimated pools and evaluate them using the actual retrieval procedure. We then select the assignment producing the largest candidate set.

Importantly, occurrences of the same value in different query attributes are treated as distinct assignments. Although they have the same inverted-index estimate, they are evaluated separately because the associated query attribute affects semantic matching and, consequently, the resulting candidate set.

⁶<https://github.com/megagonlabs/starmie>

Table 4: Average input size for Dinkelbach’s algorithm across different Top- n settings and queries.

Dataset	Top- n					
	5K	10K	15K	20K	25K	30K
SANTOS-LARGE	160	209	236	251	258	259
WDC-10K	387	618	776	853	877	881
WDC-100K	808	1245	1588	1929	2264	2518
WDC-1M	1071	1898	2841	3559	4355	5133

Table 5: Benefit of Stage 1 on the scalability benchmarks. For WDC-1M, we report results over 66 queries, as one query does not complete all pipeline stages.

Dataset	D (Input Size)	Two-stage (ms)	No S1 (ms)	Saved (%)	Speedup
SANTOS-LARGE	259	43.2	390.5	88.9%	9×
WDC-10K	881	12.6	238.4	94.7%	18.9×
WDC-100K	2518	13.4	470	97.1%	35.1×
WDC-1M	5258	20.1	1193.1	98.3%	59.3×

Query cohort. We retain queries with at least 60 candidates after inverted index retrieval, i.e., $|D| \geq 60$. This threshold supports all configurations considered in our scalability experiments:

$$k \in \{10, 15\}, \quad \alpha \in \{2, 3, 4\},$$

for which the largest required candidate pool is $\alpha \times k = 60$. Applying this criterion retains 47 of the 78 eligible queries.

Selecting the remaining parameters. We next fix the selected (distribution attribute, target value) assignments and perform a grid search over the distributional parameters to identify a setting for which the complete pipeline remains feasible for nearly all retained queries and all six (k, α) configurations. Based on this search, we set $\tau = 0.052$. Under this setting, 46 of the 47 retained queries admit a feasible solution across all six configurations. We therefore use these 46 queries as the final scalability cohort. This controlled parametrization ensures that all stages of the pipeline are exercised.

A.4 WDC Detailed Results

Table 6: Stage-1 statistics across WDC tiers for varying top-n, with $\alpha = 10$ and $k = 20$.

Tier	Metric	top-n					
		5k	10k	15k	20k	25k	30k
WDC-10K	Time (ms)	0.67	1.26	1.53	1.72	1.80	1.88
	Iterations	2.16	2.48	2.48	2.57	2.57	2.58
WDC-100K	Time (ms)	1.45	2.25	3.30	4.08	5.33	6.44
	Iterations	1.99	2.22	2.31	2.37	2.42	2.51
WDC-1M	Time (ms)	2.17	4.82	6.77	7.87	11.04	11.56
	Iterations	1.58	1.77	1.85	1.88	1.86	1.97

Table 7: Mean Stage-2 runtime (ms) across WDC tiers and pool sizes. Feasibility is reported as the number of feasible queries out of 67.

Tier	Component	Candidate Pool Size ($\alpha \times k$)			
		25	50	100	200
WDC-10K	LP Pre-check	0.873	1.071	0.994	1.202
	ILP Solver	1.426	1.583	1.697	2.162
	Total	2.299	2.654	2.691	3.364
	Feasible	67/67	67/67	67/67	67/67
WDC-100K	LP Pre-check	0.992	1.183	1.135	1.376
	ILP Solver	2.322	2.148	3.221	4.341
	Total	3.314	3.331	4.357	5.717
	Feasible	67/67	67/67	67/67	67/67
WDC-1M	LP Pre-check	0.976	1.682	1.201	1.354
	ILP Solver	1.706	2.153	2.210	2.705
	Total	2.682	3.835	3.411	4.059
	Feasible	60/67	60/67	60/67	60/67